

LAPORAN MILESTONE 4 TUGAS BESAR IF2224 TEORI BAHASA FORMAL DAN OTOMATA

ARION COMPILER

Intermediate Code and Interpreter

DEADLINE: Sabtu, 6 Juni 2026 Pukul 23:59 WIB



Dipersiapkan oleh:

Kelompok AZZAM | Kode MAR | Kelas 01

13524017	Aziza Dharma Putri
13424053	Ahmad Zaky Robbani
13524063	Marcel Luther Sitorus
13524081	Alya Nur Rahmah

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

DAFTAR ISI

DAFTAR ISI.....	2
TEORI SINGKAT.....	3
1.1. Compiler dan Tahapan Kompilasi.....	3
1.2. Decorated AST (Abstract Syntax tree).....	3
1.3. Intermediate Representation (IR).....	4
1.4. Three-Address Code (TAC).....	4
1.5. Translasi Struktur Control Flow.....	5
1.6. Interpreter sebagai Virtual Machine.....	6
1.7. Manajemen Memori Berbasis Stack.....	6
PERANCANGAN DAN IMPLEMENTASI.....	7
2.1. Perancangan Umum.....	7
2.7. Implementasi.....	13
2.7.1. Alur Program.....	13
2.7.2. Kode Program Utama.....	14
2.7.3. Kode Code Generator.....	20
2.7.4. Kode Instruction.....	27
2.7.5. Kode Interpreter.....	31
2.7.6. Kode Stack Machine.....	35
PENGUJIAN.....	40
3.1. Pengujian Pertama.....	40
3.2. Pengujian Kedua.....	59
3.3. Pengujian Ketiga.....	61
3.4. Pengujian Keempat.....	63
3.5. Pengujian Kelima.....	65
3.6. Pengujian Keenam.....	68
3.7. Pengujian Ketujuh.....	71
3.8. Pengujian Kedelapan.....	74
3.9. Pengujian Kesembilan.....	77
3.10. Pengujian Kesebelas.....	82
3.11. Pengujian Kesebelas.....	86
3.12. Pengujian Keduabelas.....	92
3.13. Pengujian Ketigabelas.....	95
3.14. Pengujian Keempatbelas.....	97
3.15. Pengujian Kelimabelas.....	100
3.16. Pengujian Keenambelas.....	102
3.17. Pengujian Ketujuhbelas.....	106
3.18. Pengujian Kedelapanbelas.....	109
KESIMPULAN DAN SARAN.....	113
4.1. Kesimpulan.....	113
4.2. Saran.....	113
LAMPIRAN.....	115
DAFTAR PUSTAKA.....	116

BAB I

TEORI SINGKAT

1.1. Compiler dan Tahapan Kompilasi

Compiler merupakan perangkat lunak yang berfungsi menerjemahkan program yang ditulis menggunakan bahasa tingkat tinggi menjadi bentuk lain yang dapat dipahami dan dieksekusi oleh komputer. Dalam implementasinya, compiler modern umumnya dibagi menjadi dua bagian utama, yaitu Front-end dan Back-end.

Front-end bertugas melakukan analisis terhadap source code yang ditulis oleh pengguna. Tahapan ini meliputi proses Lexical Analysis, Syntax Analysis, dan Semantic Analysis. Pada Lexical Analysis, rangkaian karakter pada source code diubah menjadi token-token yang memiliki makna tertentu. Selanjutnya pada Syntax Analysis, token-token tersebut disusun menjadi struktur hierarkis berdasarkan aturan grammar bahasa yang digunakan. Setelah itu dilakukan Semantic Analysis untuk memastikan bahwa program tidak hanya benar secara sintaksis, tetapi juga benar secara makna, seperti validitas penggunaan variabel, kompatibilitas tipe data, dan aturan lingkup (scope). Hasil akhir dari Front-end bukan lagi berupa teks program, melainkan representasi internal program yang telah diverifikasi. Pada tugas besar ini, representasi tersebut berupa Decorated Abstract Syntax Tree (Decorated AST) yang telah dilengkapi dengan informasi semantik serta terhubung dengan Symbol Table.

Berbeda dengan Front-end, Back-end tidak lagi berurusan dengan bentuk sintaksis bahasa pemrograman. Back-end hanya berfokus pada bagaimana representasi internal program dapat diterjemahkan menjadi instruksi yang dapat dijalankan. Oleh karena itu, Back-end bertanggung jawab terhadap proses pembangkitan kode (code generation), optimasi, pengelolaan memori, dan eksekusi program.

Pemisahan antara Front-end dan Back-end merupakan salah satu prinsip penting dalam perancangan compiler modern. Dengan adanya pemisahan tersebut, perubahan pada sintaks bahasa hanya memerlukan modifikasi pada Front-end tanpa harus mengubah mekanisme eksekusi yang terdapat pada Back-end. Sebaliknya, perubahan pada metode eksekusi atau target platform dapat dilakukan tanpa memengaruhi proses analisis sintaksis maupun semantik yang telah dibangun sebelumnya.

1.2. Decorated AST (Abstract Syntax tree)

Abstract Syntax Tree (AST) merupakan representasi struktur program yang berfokus pada hubungan semantik antar komponen program. Berbeda dengan Parse Tree yang masih mempertahankan seluruh informasi grammar, AST hanya menyimpan informasi yang relevan untuk proses analisis dan pembangkitan kode. Pada tahap Semantic Analysis, AST akan diperkaya dengan berbagai informasi tambahan sehingga menghasilkan struktur yang disebut sebagai Decorated Abstract Syntax Tree atau Decorated AST. Informasi tambahan tersebut dapat berupa tipe data hasil evaluasi suatu ekspresi, referensi ke entri pada Symbol Table, level scope tempat suatu identifier dideklarasikan, serta berbagai atribut lain yang diperlukan pada tahap selanjutnya.

Decorated AST memiliki peranan yang sangat penting karena menjadi sumber informasi utama pada proses pembangkitan kode. Setiap node dalam pohon tidak hanya merepresentasikan operasi tertentu, tetapi juga membawa informasi yang diperlukan untuk menentukan instruksi apa yang harus dihasilkan. Sebagai contoh, ketika generator menemukan operasi penjumlahan antara dua operand, informasi tipe data yang tersimpan pada node memungkinkan generator menentukan apakah operasi tersebut merupakan penjumlahan numerik, operasi logika, atau bahkan konkatenasi string. Tanpa adanya informasi semantik tersebut, proses pembangkitan kode tidak dapat dilakukan secara tepat.

Dengan kata lain, Decorated AST berfungsi sebagai jembatan antara tahap analisis program dan tahap eksekusi program. Seluruh informasi yang telah dihitung pada Semantic Analysis dimanfaatkan kembali sehingga tidak perlu dilakukan pencarian ulang terhadap deklarasi simbol maupun inferensi tipe data.

1.3. Intermediate Representation (IR)

Intermediate Representation (IR) merupakan representasi perantara yang digunakan untuk menjembatani bahasa tingkat tinggi dengan bentuk instruksi yang lebih dekat dengan mesin. Keberadaan IR memungkinkan compiler memisahkan proses analisis program dari proses eksekusi atau pembangkitan kode akhir.

Pada bahasa tingkat tinggi, suatu program direpresentasikan dalam bentuk yang mudah dipahami manusia. Sebaliknya, komputer membutuhkan instruksi yang sederhana dan terstruktur agar dapat dieksekusi secara efisien. Oleh karena itu, diperlukan suatu bentuk representasi yang mampu mempertahankan makna program sekaligus mempermudah proses transformasi menuju tahap eksekusi.

Penggunaan IR memberikan berbagai keuntungan. Pertama, IR menyediakan format yang relatif independen terhadap bahasa sumber maupun arsitektur target. Kedua, IR memungkinkan dilakukannya optimasi program sebelum tahap eksekusi. Ketiga, IR menyederhanakan proses implementasi compiler karena berbagai bahasa pemrograman dapat diterjemahkan ke dalam bentuk representasi yang sama.

Dalam banyak sistem compiler modern, IR menjadi komponen penting yang menghubungkan hasil analisis semantik dengan mekanisme eksekusi program. Dengan demikian, IR berperan sebagai fondasi utama pada tahap code generation maupun interpretasi program.

1.4. Three-Address Code (TAC)

Three-Address Code (TAC) merupakan salah satu bentuk Intermediate Representation yang paling umum digunakan dalam implementasi compiler. TAC dirancang agar setiap instruksi hanya melakukan satu operasi sederhana sehingga mudah diproses oleh mesin eksekusi. Secara umum, satu instruksi TAC memiliki bentuk:

$$hasil := operand_1 \operatorname{operator} operand_2$$

Pada bentuk tersebut, satu instruksi hanya melibatkan paling banyak tiga alamat, yaitu dua operand sumber dan satu lokasi penyimpanan hasil. Oleh karena itu representasi ini disebut sebagai Three-Address Code.

Prinsip utama TAC adalah memecah ekspresi kompleks menjadi serangkaian operasi sederhana. Sebagai contoh, ekspresi:

$$x := (a + b) \times (c - d)$$

tidak dapat langsung dieksekusi sebagai satu instruksi tunggal. Sebaliknya, ekspresi tersebut harus dipecah menjadi beberapa langkah yang lebih sederhana:

$$t1 := a + b$$

$$t2 := c - d$$

$$t3 := t1 * t2$$

$$x := t3$$

Variabel seperti $t1$, $t2$, dan $t3$ disebut sebagai temporary variables. Variabel sementara digunakan untuk menyimpan hasil perhitungan sementara yang nantinya akan digunakan oleh instruksi berikutnya. Pendekatan ini meniru cara kerja prosesor sesungguhnya yang melakukan perhitungan secara bertahap. Dengan menggunakan TAC, struktur pohon yang kompleks dapat diubah menjadi daftar instruksi linier yang lebih mudah diinterpretasikan.

1.5. Translasi Struktur Control Flow

Selain ekspresi aritmatika, bahasa pemrograman juga menyediakan berbagai struktur kontrol seperti percabangan dan perulangan. Struktur tersebut secara alami membentuk hierarki dan hubungan bersarang (nested structure) sehingga tidak dapat langsung direpresentasikan sebagai urutan instruksi linier. Untuk mengatasi permasalahan tersebut, digunakan mekanisme label dan jump instruction. Label merupakan penanda lokasi tertentu dalam daftar instruksi TAC. Label berfungsi sebagai tujuan perpindahan alur eksekusi program. Sementara itu, jump instruction digunakan untuk memindahkan eksekusi ke label tertentu.

Terdapat dua jenis jump yang umum digunakan, yaitu:

- Conditional Jump, yaitu perpindahan yang hanya dilakukan jika kondisi tertentu terpenuhi.
- Unconditional Jump, yaitu perpindahan tanpa syarat ke suatu label tertentu.

Pada struktur IF-ELSE, kondisi dievaluasi terlebih dahulu. Jika kondisi bernilai salah, eksekusi langsung berpindah ke blok ELSE. Sebaliknya, jika kondisi bernilai benar, eksekusi berlanjut ke blok THEN. Pada struktur WHILE, kondisi dievaluasi pada setiap iterasi. Jika kondisi masih bernilai benar, eksekusi kembali ke awal perulangan. Jika kondisi bernilai salah, eksekusi keluar dari perulangan. Dengan menggunakan kombinasi label dan jump instruction, struktur kontrol yang semula berbentuk pohon dapat diratakan menjadi daftar instruksi linier tanpa kehilangan makna semantiknya. Teknik ini merupakan fondasi utama dalam implementasi compiler maupun interpreter modern.

1.6. Interpreter sebagai Virtual Machine

Interpreter merupakan komponen yang bertugas mengeksekusi Intermediate Code secara langsung. Berbeda dengan compiler tradisional yang menghasilkan file executable, interpreter membaca dan menjalankan instruksi satu per satu selama program berjalan.

Secara konseptual, interpreter dapat dipandang sebagai sebuah Virtual Machine (VM). Virtual machine tersebut menyediakan lingkungan eksekusi yang mensimulasikan perilaku komputer nyata, termasuk pengelolaan memori, penyimpanan variabel, serta pengendalian alur program. Pada saat eksekusi berlangsung, interpreter menggunakan sebuah penunjuk yang disebut Instruction Pointer (IP) atau Program Counter (PC). Penunjuk ini menyimpan posisi instruksi yang sedang dieksekusi.

Siklus kerja interpreter umumnya terdiri atas tiga tahapan utama:

- Fetch, yaitu membaca instruksi yang ditunjuk oleh Instruction Pointer.
- Decode, yaitu menganalisis jenis instruksi beserta operand yang digunakan.
- Execute, yaitu menjalankan operasi sesuai instruksi yang telah dibaca.

Setelah suatu instruksi selesai dieksekusi, Instruction Pointer akan diperbarui sehingga menunjuk ke instruksi berikutnya atau ke lokasi lain apabila terjadi operasi lompatan (jump). Proses tersebut berlangsung secara berulang hingga seluruh instruksi selesai dijalankan.

1.7. Manajemen Memori Berbasis Stack

Dalam implementasi interpreter, salah satu struktur data yang paling penting adalah stack. Stack merupakan struktur data yang mengikuti prinsip Last In First Out (LIFO), yaitu elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Penggunaan stack sangat erat kaitannya dengan proses evaluasi ekspresi. Ketika suatu operand ditemukan, nilainya dapat disimpan sementara ke dalam stack. Saat operator dievaluasi, operand yang diperlukan diambil dari stack, kemudian hasil operasi dikembalikan kembali ke stack. Selain digunakan untuk evaluasi ekspresi, stack juga memainkan peranan penting dalam pemanggilan prosedur dan fungsi. Ketika suatu fungsi dipanggil, interpreter perlu menyimpan informasi seperti parameter, variabel lokal, dan alamat pengembalian (return address). Informasi tersebut disimpan dalam activation record atau stack frame yang ditempatkan pada stack. Melalui mekanisme ini, interpreter dapat menangani pemanggilan fungsi secara bertingkat maupun rekursif tanpa kehilangan konteks eksekusi sebelumnya. Oleh karena itu, stack menjadi salah satu komponen utama yang memungkinkan virtual machine menjalankan program secara benar dan efisien.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Perancangan Umum

Pada Milestone 4, Arion Compiler dikembangkan hingga tahap backend compiler dan runtime execution. Tahap ini merupakan kelanjutan dari Milestone 1 sampai Milestone 3 yang telah mencakup lexical analysis, syntax analysis, dan semantic analysis. Hasil utama dari Milestone 3 berupa Decorated Abstract Syntax Tree (Decorated AST) dan Symbol Table digunakan sebagai input utama pada tahap backend Milestone 4.

Backend pada Milestone 4 bertugas menerjemahkan Decorated AST menjadi intermediate code berbasis stack machine. Intermediate code tersebut tidak berupa executable native seperti hasil kompilasi C/C++, melainkan berupa daftar instruksi virtual machine yang dapat dijalankan oleh interpreter. Dengan demikian, hasil akhir Milestone 4 lebih menyerupai interpreter berbasis file, yaitu program Arion dapat dijalankan dari source code hingga langsung menghasilkan output runtime melalui mode eksekusi yang disediakan.

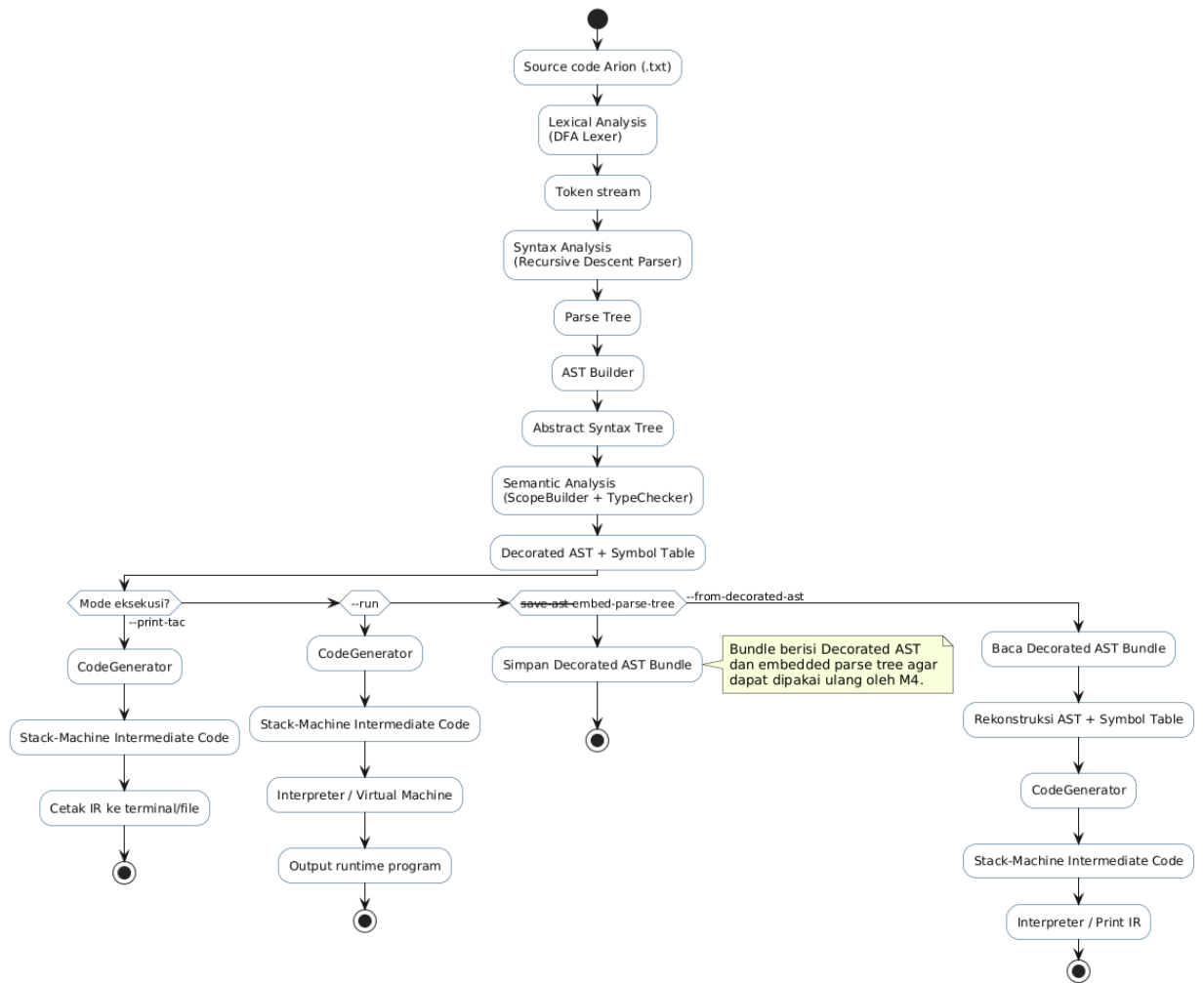
Secara umum, pipeline Arion Compiler sampai Milestone 4 terdiri atas beberapa tahapan. Source code Arion pertama-tama diproses oleh lexer untuk menghasilkan token. Token kemudian diproses oleh parser untuk menghasilkan parse tree. Parse tree diterjemahkan menjadi AST dan dianalisis oleh semantic analyzer untuk menghasilkan Decorated AST dan Symbol Table. Setelah itu, Decorated AST diteruskan ke CodeGenerator untuk menghasilkan intermediate code. Intermediate code yang dihasilkan kemudian dijalankan oleh Interpreter menggunakan StackMachine hingga menghasilkan output program.

Fokus utama Milestone 4 berada pada tiga komponen backend, yaitu CodeGenerator, InstructionProgram, dan Interpreter. CodeGenerator bertugas melakukan traversal terhadap Decorated AST dan menghasilkan instruksi stack machine. InstructionProgram berfungsi sebagai container intermediate code yang menyimpan daftar instruksi, string pool, dan real pool. Interpreter menjalankan intermediate code menggunakan model virtual machine berbasis stack machine dengan siklus fetch-decode-execute.

Pada implementasi final, compiler menyediakan beberapa mode eksekusi untuk mendukung kebutuhan Milestone 4. Mode `--print-tac` digunakan untuk menampilkan intermediate code hasil code generation. Mode `--run` digunakan untuk menjalankan program Arion dari source code hingga menghasilkan output runtime. Mode `--run-ir` digunakan untuk menjalankan intermediate code dari file secara langsung. Selain itu, mode `--from-decorated-ast` digunakan untuk membaca kembali bundle Decorated AST sebagai input backend Milestone 4.

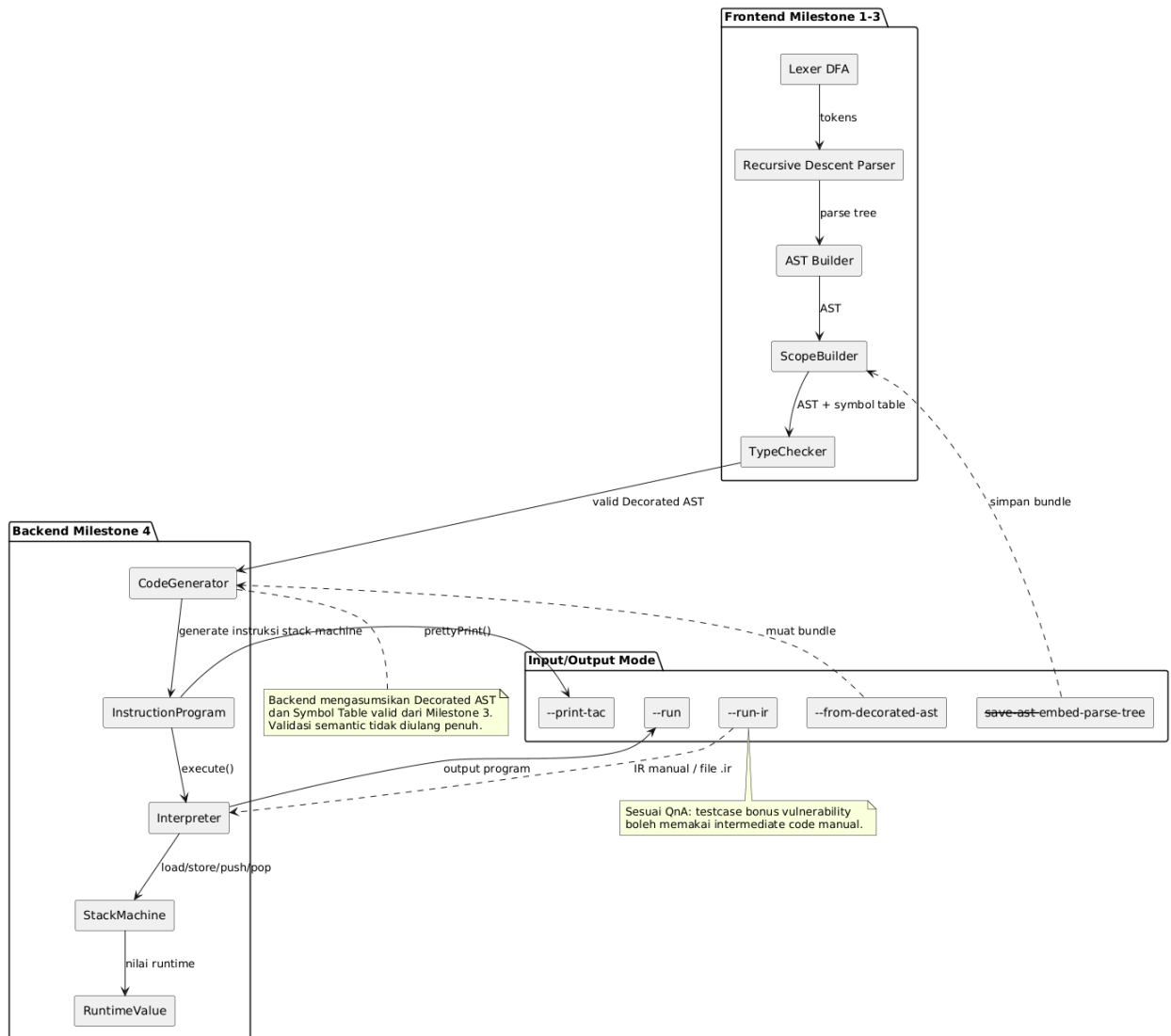
Dengan rancangan tersebut, Arion Compiler tidak hanya melakukan analisis terhadap source code, tetapi juga mampu menghasilkan intermediate code dan mengeksekusinya menggunakan interpreter. Pendekatan ini sesuai dengan klarifikasi QnA bahwa hasil akhir Milestone 4 adalah interpreter, bukan compiler native yang menghasilkan file executable.

Arion Compiler Milestone 4 - Pipeline Eksekusi



Gambar 2.1 Pipeline Milestone 4 Arion Compiler

Arsitektur Backend Milestone 4



Gambar 2.2 Arsitektur Komponen Backend Milestone 4

2.2. Asumsi Input Decorated AST

Pada Milestone 4, input utama backend adalah Decorated AST dan Symbol Table yang dihasilkan dari Milestone 3. Berdasarkan klarifikasi QnA, backend tidak diwajibkan melakukan validasi ulang penuh terhadap Decorated AST dan Symbol Table. Oleh karena itu, implementasi Milestone 4 mengasumsikan bahwa Decorated AST yang diterima oleh CodeGenerator sudah valid secara sintaksis dan semantik.

Validasi seperti undeclared identifier, duplicate declaration, type mismatch, invalid array index type, invalid record field, dan kesalahan semantic lainnya merupakan tanggung jawab Milestone 3. Tahap semantic analysis sebelumnya telah melakukan pemeriksaan scope, type checking, validasi deklarasi, validasi penggunaan identifier, serta pembentukan Symbol Table. Dengan demikian,

backend Milestone 4 dapat berfokus pada proses translasi Decorated AST menjadi intermediate code dan eksekusi intermediate code menggunakan interpreter.

Meskipun backend tidak melakukan validasi semantik ulang secara penuh, Milestone 4 tetap melakukan validasi yang berkaitan dengan code generation dan runtime safety. CodeGenerator akan menolak node AST yang belum didukung oleh backend. Interpreter dan StackMachine juga melakukan validasi runtime seperti pengecekan alamat memori, target jump, instruction level, stack underflow, stack overflow, division by zero, integer overflow, dan array out-of-bounds.

Selain itu, untuk mendukung kebutuhan input Decorated AST, implementasi menyediakan mode `--from-decorated-ast`. Mode ini membaca bundle Decorated AST yang dihasilkan oleh compiler melalui mode `--save-ast` dengan opsi `--embed-parse-tree`. Bundle tersebut digunakan agar backend dapat dijalankan kembali dari hasil Decorated AST tanpa harus menulis ulang source code. Pendekatan ini digunakan karena Decorated AST tidak hanya berisi struktur pohon, tetapi juga membutuhkan informasi tambahan dari Symbol Table, tipe data, scope, dan atribut semantik lainnya.

Dengan asumsi tersebut, pembagian tanggung jawab antar milestone menjadi jelas. Milestone 3 bertanggung jawab menghasilkan Decorated AST dan Symbol Table yang valid, sedangkan Milestone 4 bertanggung jawab menerjemahkan Decorated AST tersebut menjadi intermediate code dan menjalankannya menggunakan interpreter berbasis stack machine.

2.3. Format Intermediate Code

Intermediate code pada Arion Compiler Milestone 4 direpresentasikan dalam bentuk instruksi stack machine. Intermediate code ini berfungsi sebagai representasi perantara antara Decorated AST dan proses eksekusi runtime pada interpreter. Dengan adanya intermediate code, interpreter tidak perlu mengeksekusi struktur AST secara langsung, melainkan cukup menjalankan daftar instruksi linear yang telah dihasilkan oleh CodeGenerator.

Walaupun dalam teori compiler intermediate code sering direpresentasikan dalam bentuk Three-Address Code (TAC) klasik seperti `t1 := a + b`, implementasi Arion Milestone 4 menggunakan bentuk stack-machine intermediate code. Bentuk ini tetap termasuk intermediate representation karena menjadi representasi perantara antara hasil semantic analysis dan proses eksekusi. Perbedaanannya, hasil operasi tidak disimpan pada temporary variable eksplisit seperti `t1` atau `t2`, melainkan disimpan sementara pada evaluation stack.

Setiap instruksi intermediate code memiliki format umum sebagai berikut:

`<alamat> <opcode> <level> <operand>`

Komponen alamat menunjukkan posisi instruksi dalam InstructionProgram. Opcode menunjukkan jenis operasi yang harus dijalankan oleh interpreter. Level digunakan untuk menyimpan informasi lexical level atau kebutuhan khusus instruksi tertentu. Operand digunakan sebagai nilai tambahan,

seperti literal integer, alamat variabel, target lompatan, indeks string pool, indeks real pool, atau kode operasi tambahan pada instruksi OPR.

Contoh intermediate code sederhana adalah sebagai berikut:

```
0 INT 0 4
1 LIT 0 10
2 STO 0 3
3 LOD 0 3
4 OPR 0 14 ; WRTLN
5 RET 0 0
```

Instruksi tersebut menunjukkan proses eksekusi program sederhana. Instruksi INT mengalokasikan frame runtime. Instruksi LIT memasukkan literal integer 10 ke evaluation stack. Instruksi STO menyimpan nilai tersebut ke alamat variabel. Instruksi LOD memuat kembali nilai variabel ke stack. Instruksi OPR dengan operasi WRTLN mencetak nilai tersebut ke output, sedangkan RET mengakhiri eksekusi program.

Selain daftar instruksi, InstructionProgram juga dapat menyimpan pool tambahan seperti string pool dan real pool. Pool digunakan agar literal kompleks seperti string dan real tidak perlu disimpan langsung di dalam operand instruksi. Sebagai gantinya, instruksi seperti LITS dan LITR membawa indeks menuju pool yang sesuai. Dengan pendekatan ini, format instruksi tetap sederhana dan konsisten, yaitu opcode, level, dan operand.

Format intermediate code berbasis stack machine dipilih karena sesuai dengan proses traversal AST. Pada ekspresi biner, CodeGenerator cukup menghasilkan instruksi untuk operand kiri, lalu operand kanan, kemudian menghasilkan instruksi operasi. Interpreter akan mengambil operand dari evaluation stack, menjalankan operasi, dan mengembalikan hasilnya ke stack. Pendekatan ini membuat translasi ekspresi bertingkat menjadi lebih sederhana dan konsisten.

2.4. Instruction Set Stack Machine

Instruction set pada Milestone 4 dirancang untuk mendukung proses eksekusi program Arion menggunakan interpreter berbasis stack machine. Sebagian instruksi mengikuti instruksi dasar pada spesifikasi, seperti INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, dan RET. Selain itu, implementasi juga menambahkan beberapa instruksi tambahan untuk mendukung fitur runtime yang lebih lengkap, seperti literal boolean, literal real, literal string, array access, record field access, indirect load/store, dan runtime bounds checking.

Penambahan instruksi dilakukan agar intermediate code tetap sederhana dan interpreter dapat menangani fitur bahasa Arion secara modular. Instruksi tambahan tetap mengikuti format yang sama, yaitu opcode, level, dan operand, sehingga tidak mengubah struktur dasar InstructionProgram.

Instruksi	Fungsi
INT	Mengalokasikan frame runtime pada stack machine
LIT	Memasukkan literal integer ke evaluation stack
LITB	Memasukkan literal boolean ke evaluation stack
LITR	Memasukkan literal real dari real pool ke evaluation stack
LITS	Memasukkan literal string dari string pool ke evaluation stack
LOD	Mengambil nilai variabel dari runtime memory
STO	Menyimpan nilai dari evaluation stack ke runtime memory
ADDR	Menghasilkan alamat variabel, elemen array, atau field record
LODI	Mengambil nilai secara indirect dari alamat yang tersimpan di stack
STOI	Menyimpan nilai secara indirect ke alamat yang tersimpan di stack
CAL	Memanggil procedure atau function
RET	Mengakhiri program atau kembali dari procedure/function
JMP	Melakukan lompatan tanpa syarat ke alamat instruksi tertentu
JPC	Melakukan lompatan jika kondisi bernilai false
CHK	Melakukan runtime bounds checking, terutama untuk akses array
OPR	Menjalankan operasi aritmatika, relasional, input, dan output
POP	Membuang nilai teratas dari evaluation stack

Instruksi LOD dan STO digunakan untuk akses variabel biasa yang alamatnya sudah diketahui pada saat code generation. Untuk akses yang alamatnya perlu dihitung saat runtime, seperti elemen array dan field record, digunakan instruksi ADDR, LODI, dan STOI. Instruksi ADDR menghasilkan alamat runtime, LODI membaca nilai dari alamat tersebut, sedangkan STOI menyimpan nilai ke alamat tersebut.

Instruksi CHK digunakan sebagai mekanisme runtime safety untuk memastikan indeks array berada dalam batas yang valid. Jika indeks berada di luar batas, interpreter akan menghasilkan runtime error. Dengan demikian, sebagian pengecekan array dilakukan pada semantic analysis, sedangkan validasi batas aktual pada saat eksekusi dilakukan oleh interpreter.

Instruksi OPR digunakan sebagai instruksi umum untuk operasi yang bersifat aritmatika, relasional, input, dan output. Operand pada instruksi OPR menunjukkan sub-operasi yang harus dijalankan oleh interpreter.

OPR	Fungsi
NEG	Operasi unary minus
ADD	Penjumlahan
SUB	Pengurangan
MUL	Perkalian
DIV	Pembagian
MOD	Modulo
EQL	Perbandingan sama dengan
NEQ	Perbandingan tidak sama dengan
LSS	Perbandingan kurang dari
LEQ	Perbandingan kurang dari atau sama dengan
GTR	Perbandingan lebih dari
GEQ	Perbandingan lebih dari atau sama dengan
WRT	Mencetak nilai tanpa newline
WRTLN	Mencetak nilai dengan newline
RDI	Membaca input dari pengguna
RDLN	Menangani akhir baris input untuk readln
POP	Membuang nilai dari evaluation stack apabila direpresentasikan sebagai sub-operasi OPR

Dengan instruction set tersebut, intermediate code Arion dapat merepresentasikan berbagai fitur bahasa, mulai dari assignment sederhana, operasi ekspresi, percabangan, perulangan, procedure/function call, array access, record access, hingga input/output. Selain itu, beberapa instruksi juga dirancang untuk mendukung runtime safety, seperti validasi alamat, validasi target jump, validasi stack, dan pengecekan batas array.

2.5. Runtime Value dan Memory Model

RuntimeValue digunakan untuk merepresentasikan nilai yang hidup selama proses eksekusi intermediate code. Pada interpreter berbasis stack machine, setiap nilai yang diproses oleh instruksi seperti LIT, LOD, LITB, LITR, LITS, dan OPR disimpan dalam bentuk RuntimeValue. Dengan adanya RuntimeValue, interpreter dapat membedakan jenis nilai yang sedang diproses, sehingga operasi runtime dapat dilakukan sesuai tipe datanya.

Pada implementasi final Milestone 4, RuntimeValue mendukung beberapa tipe utama, yaitu Integer, Real, Boolean, Character, dan String. Tipe Integer digunakan untuk operasi bilangan bulat, alamat runtime, indeks array, serta operasi aritmatika dasar. Tipe Real digunakan untuk nilai bilangan pecahan. Tipe Boolean digunakan untuk hasil operasi relasional dan kondisi control flow. Tipe Character dan String digunakan untuk assignment, ekspresi, serta operasi input/output.

RuntimeValue tidak hanya menyimpan nilai aktual, tetapi juga informasi tipe dari nilai tersebut. Informasi tipe ini diperlukan agar interpreter dapat menentukan operasi yang valid. Sebagai contoh, operasi ADD pada dua nilai Integer akan menghasilkan RuntimeValue bertipe Integer, sedangkan operasi relasional seperti LSS atau GEQ akan menghasilkan RuntimeValue bertipe Boolean. Jika terjadi operasi yang tidak sesuai, interpreter dapat menghasilkan runtime error sesuai mekanisme validasi yang tersedia.

Dalam eksekusi program, nilai runtime disimpan pada dua struktur utama, yaitu evaluation stack dan runtime memory. Evaluation stack digunakan sebagai tempat penyimpanan sementara ketika interpreter menjalankan ekspresi. Instruksi LIT, LITB, LITR, LITS, dan LOD akan memasukkan nilai ke stack. Instruksi OPR kemudian mengambil operand dari stack, menjalankan operasi, dan memasukkan hasilnya kembali ke stack. Setelah ekspresi selesai dihitung, instruksi STO atau STOI dapat menyimpan hasil tersebut ke runtime memory.

Runtime memory digunakan untuk menyimpan nilai variabel, parameter, nilai sementara frame, elemen array, dan field record. Akses ke runtime memory dilakukan melalui alamat yang dihasilkan oleh CodeGenerator. Untuk variabel sederhana, instruksi LOD dan STO dapat digunakan secara langsung. Untuk struktur data yang membutuhkan perhitungan alamat saat runtime, seperti array dan record, digunakan instruksi ADDR, LODI, dan STOI.

Selain runtime memory, InstructionProgram juga menyimpan literal pool, yaitu string pool dan real pool. String pool digunakan untuk menyimpan literal string yang muncul pada program, sedangkan real pool digunakan untuk menyimpan literal real. Instruksi LITS membawa indeks menuju string pool, sementara instruksi LITR membawa indeks menuju real pool. Dengan pendekatan ini, operand instruksi tetap berbentuk integer sehingga format intermediate code tetap sederhana dan konsisten.

Memory model pada interpreter juga dilengkapi dengan mekanisme validasi runtime. Interpreter dan StackMachine melakukan pengecekan terhadap stack underflow, stack overflow, invalid address, invalid jump target, invalid instruction, division by zero, integer overflow, dan array

out-of-bounds. Validasi ini diperlukan karena Milestone 4 tidak hanya menghasilkan intermediate code, tetapi juga menjalankan intermediate code tersebut secara langsung.

Komponen Runtime	Fungsi
RuntimeValue	Merepresentasikan nilai runtime beserta tipenya
Evaluation stack	Menyimpan operand dan hasil sementara saat ekspresi dieksekusi
Runtime memory	Menyimpan nilai variabel, parameter, array, record, dan frame runtime
String pool	Menyimpan literal string yang digunakan oleh instruksi LITS
Real pool	Menyimpan literal real yang digunakan oleh instruksi LITR
Instruction pointer	Menunjukkan instruksi yang sedang dieksekusi
Stack frame	Menyimpan konteks eksekusi procedure/function
Runtime validation	Menangani error seperti invalid address, stack underflow, dan array out-of-bounds

Dengan model tersebut, interpreter dapat menjalankan program Arion secara bertahap dari intermediate code. Setiap ekspresi dievaluasi melalui stack, setiap variabel disimpan pada runtime memory, dan literal kompleks seperti string serta real dikelola melalui pool. Pendekatan ini membuat desain interpreter lebih modular dan memudahkan proses validasi runtime.

2.6. Code Generator

CodeGenerator merupakan komponen backend yang bertugas menerjemahkan Decorated AST menjadi intermediate code berbasis stack machine. Proses translasi dilakukan dengan menelusuri node-node pada Decorated AST dan menghasilkan instruksi yang sesuai untuk setiap jenis statement, expression, declaration, dan control flow.

Input utama CodeGenerator adalah Decorated AST dan Symbol Table yang dihasilkan oleh Milestone 3. Decorated AST menyediakan struktur program, sedangkan Symbol Table menyediakan informasi semantik seperti nama identifier, jenis simbol, tipe data, lexical level, alamat relatif, ukuran variabel, serta hubungan antar scope. Informasi ini digunakan oleh CodeGenerator untuk menentukan instruksi yang harus dihasilkan, alamat runtime yang digunakan, serta ukuran frame yang perlu dialokasikan.

CodeGenerator bekerja menggunakan pendekatan traversal rekursif. Untuk setiap node AST, CodeGenerator memanggil fungsi generation yang sesuai dengan jenis node tersebut. Pada node

statement, CodeGenerator menghasilkan instruksi yang mengubah state runtime, seperti assignment, procedure call, percabangan, dan perulangan. Pada node expression, CodeGenerator menghasilkan instruksi yang memasukkan nilai ke evaluation stack dan menjalankan operasi melalui instruksi OPR.

Pada expression biner, CodeGenerator terlebih dahulu menghasilkan instruksi untuk operand kiri, kemudian operand kanan, lalu menghasilkan instruksi OPR sesuai operator yang digunakan. Sebagai contoh, ekspresi $a + b$ akan diterjemahkan menjadi instruksi untuk memuat nilai a , instruksi untuk memuat nilai b , kemudian instruksi OPR ADD. Hasil operasi tersebut akan berada di evaluation stack dan dapat digunakan oleh statement lain, misalnya assignment atau output.

Pada assignment, CodeGenerator membedakan antara assignment ke variabel sederhana dan assignment ke lokasi yang membutuhkan alamat tidak langsung. Untuk variabel sederhana, nilai ekspresi disimpan menggunakan instruksi STO. Untuk elemen array atau field record, CodeGenerator menghasilkan alamat tujuan terlebih dahulu, kemudian menyimpan nilai menggunakan instruksi STOI. Pendekatan ini membuat akses struktur data kompleks tetap dapat direpresentasikan dalam instruction set stack machine.

Pada control flow, CodeGenerator menggunakan instruksi JMP dan JPC. Untuk statement if, CodeGenerator menghasilkan instruksi pengecekan kondisi, JPC menuju cabang else atau akhir statement, instruksi untuk then branch, dan jika diperlukan JMP untuk melewati else branch. Untuk loop seperti while, for, dan repeat-until, CodeGenerator menyimpan alamat awal loop, menghasilkan instruksi pengecekan kondisi, lalu melakukan backpatch terhadap target lompatan setelah seluruh instruksi loop selesai dihasilkan.

Pada procedure dan function call, CodeGenerator menghasilkan instruksi untuk menyiapkan argumen, melakukan pemanggilan dengan CAL, dan menangani nilai hasil function apabila ada. Built-in procedure seperti read, readln, write, dan writeln diperlakukan secara khusus karena berhubungan langsung dengan input dan output interpreter. Untuk write dan writeln, CodeGenerator menghasilkan instruksi untuk mengevaluasi argumen, lalu menghasilkan operasi output melalui OPR WRT atau OPR WRTLN.

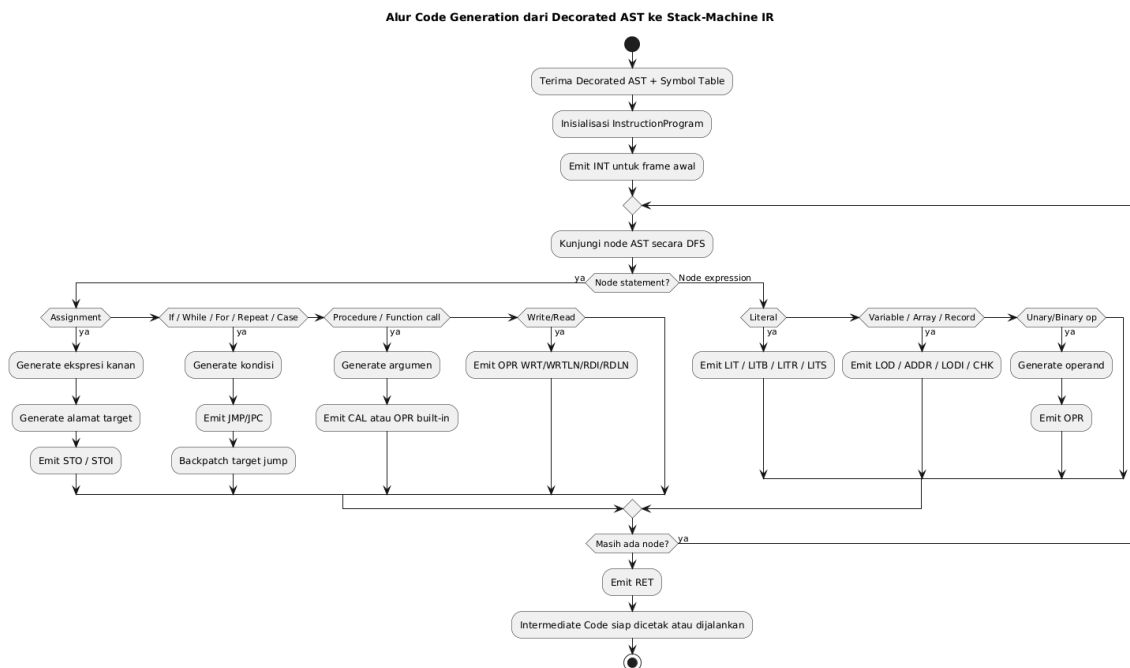
Karena input CodeGenerator diasumsikan berupa Decorated AST yang sudah valid dari Milestone 3, CodeGenerator tidak melakukan validasi semantik ulang secara penuh. Kesalahan seperti undeclared identifier, duplicate declaration, dan type mismatch sudah menjadi tanggung jawab semantic analyzer. Namun, CodeGenerator tetap melakukan validasi implementasi dengan menolak node AST yang belum didukung oleh backend. Dengan demikian, tanggung jawab CodeGenerator adalah menerjemahkan AST valid menjadi intermediate code yang dapat dieksekusi oleh interpreter.

Jenis Node AST	Strategi Code Generation
----------------	--------------------------

Program	Menghasilkan alokasi frame utama, body program, lalu instruksi RET
Declaration	Menggunakan informasi symbol table untuk menentukan alamat dan ukuran
Assignment	Menghasilkan instruksi ekspresi, lalu menyimpan hasil dengan STO atau STOI
Variable access	Menghasilkan LOD untuk variabel sederhana atau alamat untuk akses kompleks
Integer literal	Menghasilkan instruksi LIT
Boolean literal	Menghasilkan instruksi LITB
Real literal	Menyimpan literal ke real pool, lalu menghasilkan LITR
String literal	Menyimpan literal ke string pool, lalu menghasilkan LITS
Character literal	Menghasilkan runtime value bertipe character
Binary expression	Generate operand kiri, operand kanan, lalu OPR sesuai operator
Unary expression	Generate operand, lalu OPR NEG atau operasi unary lain
If statement	Menggunakan JPC, optional JMP, dan backpatch target
While loop	Menyimpan alamat awal loop, mengecek kondisi, lalu kembali ke awal
For loop	Menghasilkan inisialisasi counter, pengecekan batas, body, dan update counter
Repeat-until	Menghasilkan body terlebih dahulu, lalu mengecek kondisi di akhir
Case statement	Membandingkan ekspresi dengan label case dan menggunakan jump/backpatch
Procedure call	Menyiapkan argumen, lalu menghasilkan CAL atau operasi built-in
Function call	Menyiapkan argumen, menghasilkan CAL, dan menyimpan hasil di stack
Array access	Menghitung offset, menjalankan CHK, lalu memakai LODI/STOI

Record field access	Menghitung offset field, lalu memakai LODI/STOI
Read/readln	Menghasilkan alamat target dan operasi input
Write/writeln	Menghasilkan ekspresi argumen dan operasi output WRT/WRTLN

Tabel tersebut menunjukkan bahwa setiap node Decorated AST memiliki pola translasi tertentu ke instruction set stack machine. Dengan pola ini, CodeGenerator dapat menerjemahkan struktur program yang kompleks menjadi daftar instruksi linear yang lebih mudah dijalankan oleh interpreter.



Gambar 2.3 Alur Code Generation dari Decorated AST ke Intermediate Code

2.7. Interpreter dan Stack Machine

Interpreter merupakan komponen backend yang bertugas menjalankan intermediate code yang telah dihasilkan oleh CodeGenerator. Intermediate code pada Milestone 4 tidak dieksekusi langsung oleh sistem operasi, melainkan dijalankan oleh interpreter menggunakan model virtual machine berbasis stack machine.

Interpreter menjalankan instruksi dengan siklus fetch-decode-execute. Pada tahap fetch, interpreter membaca instruksi berdasarkan instruction pointer. Pada tahap decode, interpreter mengidentifikasi opcode, level, dan operand dari instruksi tersebut. Pada tahap execute, interpreter menjalankan operasi sesuai instruksi, seperti memasukkan nilai ke stack, membaca atau menyimpan nilai dari

runtime memory, menjalankan operasi aritmatika, melakukan lompatan control flow, atau mencetak output.

StackMachine digunakan sebagai struktur runtime utama selama proses eksekusi. Komponen ini mengelola evaluation stack dan runtime memory. Evaluation stack digunakan untuk menyimpan operand sementara pada saat ekspresi dievaluasi. Runtime memory digunakan untuk menyimpan nilai variabel, parameter, elemen array, field record, dan frame runtime procedure/function.

Instruksi seperti LIT, LITB, LITR, LITS, dan LOD akan memasukkan nilai ke evaluation stack. Instruksi OPR mengambil operand dari stack, menjalankan operasi, lalu memasukkan hasilnya kembali ke stack. Instruksi STO dan STOI menyimpan nilai dari stack ke runtime memory. Untuk control flow, instruksi JMP dan JPC mengubah instruction pointer agar eksekusi dapat berpindah ke alamat instruksi tertentu.

Interpreter juga menangani input dan output. Built-in procedure write dan writeln dijalankan melalui operasi output pada instruksi OPR. Untuk write, nilai dicetak tanpa newline, sedangkan writeln mencetak nilai dengan newline. Built-in procedure read dan readln dijalankan dengan membaca input dari stream yang disediakan, kemudian menyimpan nilai hasil input ke alamat runtime yang sesuai.

Selain menjalankan intermediate code yang valid, interpreter juga dilengkapi mekanisme runtime safety. Mekanisme ini digunakan untuk mencegah eksekusi intermediate code yang rusak atau berbahaya. Beberapa validasi yang dilakukan antara lain pengecekan stack underflow, stack overflow, invalid address, invalid jump target, invalid instruction, invalid instruction level, division by zero, integer overflow, invalid string pool index, invalid real pool index, dan array out-of-bounds.

Dengan adanya interpreter dan StackMachine, Arion Compiler dapat menjalankan program Arion dari intermediate code hingga menghasilkan output runtime. Pendekatan ini membuat hasil akhir Milestone 4 sesuai dengan model interpreter berbasis file, bukan compiler native yang menghasilkan executable.

Komponen	Fungsi
Interpreter	Menjalankan intermediate code dengan siklus fetch-decode-execute
Instruction pointer	Menunjukkan alamat instruksi yang sedang dijalankan
StackMachine	Mengelola evaluation stack dan runtime memory

Evaluation stack	Menyimpan operand dan hasil sementara saat eksekusi ekspresi
Runtime memory	Menyimpan variabel, parameter, array, record, dan frame runtime
RuntimeValue	Merepresentasikan nilai runtime beserta tipe datanya
Output buffer	Menyimpan hasil keluaran dari <code>write</code> dan <code>writeln</code>
Input stream	Menyediakan input untuk <code>read</code> dan <code>readln</code>
Runtime validation	Memastikan eksekusi aman dari error runtime

2.8. Perancangan Intermediate Code

Intermediate code yang digunakan pada milestone ini berbentuk stack-based instruction set yang menyerupai konsep P-Code Machine. Intermediate code berfungsi sebagai representasi perantara antara Decorated AST dengan proses runtime execution pada interpreter. Dengan adanya representasi intermediate tersebut, interpreter tidak perlu memahami struktur AST secara langsung karena seluruh struktur program telah diratakan menjadi daftar instruksi linear.

Setiap instruksi dirancang memiliki tiga komponen utama, yaitu opcode, lexical level, dan operand. Opcode merepresentasikan jenis operasi yang akan dijalankan interpreter, lexical level digunakan untuk kebutuhan scope dan activation record, sedangkan operand digunakan sebagai parameter tambahan seperti literal value, alamat memori, atau target lompatan. Instruksi-instruksi yang digunakan mengikuti spesifikasi milestone 4, seperti:

- LIT untuk memasukkan literal ke stack
- LOD untuk mengambil nilai variabel dari memori
- STO untuk menyimpan nilai ke memori
- INT untuk mengalokasikan memori runtime
- JMP dan JPC untuk control flow
- OPR untuk operasi aritmatika dan logika
- RET untuk mengakhiri eksekusi

Pada implementasinya, seluruh instruction disimpan secara sequential di dalam instruction program sehingga interpreter dapat membacanya menggunakan instruction pointer. Pendekatan linear instruction ini dipilih karena lebih sederhana untuk proses interpretasi dibandingkan mempertahankan struktur tree saat runtime.

Selain opcode utama, operasi-operasi aritmatika dan relasional direpresentasikan sebagai sub-operation dari instruksi OPR. Operasi seperti penjumlahan, pengurangan, perkalian, pembagian, modulo, hingga operasi relasional seperti greater-than dan equality diterjemahkan menjadi operation code tertentu. Dengan demikian interpreter cukup menangani satu jenis instruction operation tetapi tetap dapat menjalankan banyak jenis operasi berbeda.

2.9. Perancangan Intermediate Code Generator

Intermediate Code Generator dirancang untuk menerjemahkan Decorated AST menjadi daftar instruction intermediate code menggunakan pendekatan traversal berbasis Depth First Search (DFS). Pendekatan DFS dipilih karena struktur AST bersifat hierarkis sehingga proses translasi expression dan statement dapat dilakukan secara rekursif dari node terdalam menuju node induk.

Pada proses translasi, setiap jenis node AST memiliki aturan translasi tersendiri. Statement seperti assignment, if statement, while statement, dan write statement diterjemahkan menjadi kombinasi instruction yang berbeda sesuai semantic meaning dari node tersebut. Untuk assignment statement, proses translasi dilakukan dengan mengevaluasi expression di sisi kanan terlebih dahulu, kemudian hasil evaluasi tersebut disimpan ke alamat memori variabel tujuan. Expression diterjemahkan menggunakan evaluation stack sehingga operand kiri dan kanan akan dimuat terlebih dahulu sebelum operator dijalankan. Sebagai contoh, expression: $x * 2$, diterjemahkan dengan:

- Memuat nilai variabel x
- Memuat literal 2
- Menjalankan operasi perkalian
- Menyimpan hasil ke stack

Pendekatan stack evaluation ini digunakan secara konsisten pada seluruh binary expression karena mempermudah proses translasi expression bertingkat maupun nested expression.

Pada if statement, generator menggunakan conditional jump (JPC) untuk mengatur alur percabangan program. Ketika kondisi bernilai false, interpreter akan melompat menuju blok else atau keluar dari blok if. Sementara itu, jika kondisi bernilai true, eksekusi akan langsung berlanjut ke statement berikutnya menggunakan konsep fall-through sebagaimana dijelaskan pada spesifikasi milestone. Karena target lompatan belum selalu diketahui saat instruction pertama kali dibuat, generator menggunakan mekanisme backpatching. Mekanisme ini memungkinkan alamat tujuan jump diperbaiki setelah seluruh body statement selesai diterjemahkan.

Untuk while statement, generator menerapkan pola loop berbasis label awal dan backward jump. Proses translasi dimulai dengan menyimpan posisi awal loop, kemudian menghasilkan instruction evaluasi kondisi. Jika kondisi tidak memenuhi, interpreter akan melompat keluar loop menggunakan conditional jump. Sebaliknya, apabila kondisi masih memenuhi, body loop akan dijalankan dan interpreter kembali melompat ke awal loop menggunakan unconditional jump.

Selain statement kontrol, generator juga menangani translasi statement output seperti writeln. Expression yang akan dicetak terlebih dahulu dimuat ke stack, kemudian interpreter menjalankan operasi output menggunakan operation code khusus.

2.10. Perancangan Stack Machine

Interpreter pada milestone ini menggunakan arsitektur stack machine sebagai media penyimpanan runtime. Stack machine dipilih karena sesuai dengan model evaluasi expression berbasis stack serta lebih sederhana dibandingkan arsitektur register machine.

Secara umum, stack machine memiliki dua komponen utama, yaitu runtime memory dan evaluation stack. Runtime memory digunakan untuk menyimpan variabel program beserta informasi activation record seperti static link, dynamic link, dan return address. Sementara itu, evaluation stack digunakan untuk melakukan evaluasi expression dan penyimpanan temporary value selama proses eksekusi berlangsung.

Pada implementasinya, layout memori mengikuti spesifikasi milestone 4. Tiga slot pertama pada stack frame digunakan untuk: Static Link, Dynamic Link, Return Address, sedangkan slot setelahnya digunakan sebagai area penyimpanan variabel lokal program. Pendekatan ini dipilih agar sistem dapat mendukung function call dan nested scope pada pengembangan milestone selanjutnya. Dengan adanya activation record, setiap pemanggilan fungsi nantinya dapat memiliki ruang memori sendiri tanpa saling mengganggu.

Evaluation stack digunakan secara intensif selama runtime execution. Ketika interpreter menjalankan expression seperti operasi penjumlahan atau perbandingan, operand akan terlebih dahulu di-push ke stack. Setelah operasi selesai dilakukan, hasil operasi akan dikembalikan ke stack sebagai temporary result. Sebagai contoh pada operasi: $x + y$, interpreter akan:

- Push nilai x
- Push nilai y
- Pop kedua operand
- Menjalankan operasi penjumlahan
- Push hasil operasi kembali ke stack

Model ini membuat proses evaluasi expression menjadi sangat fleksibel dan konsisten untuk berbagai bentuk expression.

2.11. Perancangan Runtime Value

Nilai runtime pada interpreter direpresentasikan menggunakan abstraction runtime value agar seluruh operasi runtime dapat ditangani secara seragam. Runtime value menyimpan informasi tipe data beserta actual value yang digunakan selama proses eksekusi. Pada milestone ini, tipe data runtime yang didukung meliputi integer dan boolean

Representasi runtime value digunakan agar interpreter dapat melakukan validasi tipe saat operasi dijalankan. Meskipun bahasa Arion bersifat strongly typed dan semantic analysis telah melakukan pengecekan tipe sebelumnya, runtime representation tetap diperlukan agar interpreter dapat mengelola nilai runtime secara aman dan konsisten.

Selain itu, abstraction runtime value juga mempermudah pengembangan fitur lanjutan seperti array, string, maupun object pada milestone berikutnya karena interpreter tidak langsung bergantung pada primitive type bahasa C++.

2.12. Perancangan Interpreter

Interpreter dirancang sebagai virtual machine yang bertugas menjalankan daftar intermediate code hasil translasi AST. Interpreter bekerja menggunakan konsep instruction pointer (IP) yang menunjuk instruksi yang sedang dijalankan. Siklus kerja interpreter mengikuti konsep fetch-decode-execute cycle sebagaimana dijelaskan pada spesifikasi milestone 4. Pada tahap fetch, interpreter mengambil instruction berdasarkan posisi instruction pointer. Setelah itu instruction akan di-decode untuk menentukan jenis operasi yang harus dijalankan. Selanjutnya interpreter mengeksekusi operasi tersebut dan memperbarui posisi instruction pointer menuju instruction berikutnya.

Untuk instruction biasa seperti arithmetic operation, instruction pointer akan bergerak maju satu langkah setelah eksekusi selesai. Namun pada instruction control flow seperti JMP dan JPC, instruction pointer dapat langsung berpindah ke alamat tertentu sesuai operand instruction. Interpreter juga bertanggung jawab untuk melakukan manipulasi stack machine selama runtime execution. Operasi seperti push, pop, load variable, store variable, arithmetic evaluation, hingga control flow seluruhnya dilakukan oleh interpreter berdasarkan instruction yang sedang dijalankan.

Pada implementasinya, interpreter juga dilengkapi mekanisme error handling untuk mencegah kerusakan runtime. Mekanisme tersebut diperlukan agar interpreter tidak mengalami crash ketika menerima instruction yang tidak valid atau ketika terjadi kesalahan runtime selama eksekusi program berlangsung. Dengan adanya interpreter berbasis stack machine ini, bahasa pemrograman Arion pada milestone 4 telah mampu menjalankan program hasil translasi secara penuh mulai dari Decorated AST hingga menghasilkan output runtime akhir.

2.13. Implementasi

2.13.1. Alur Program

Pada milestone 4, alur utama program compiler Arion diperluas hingga mampu melakukan proses backend compiler berupa intermediate code generation dan runtime execution menggunakan interpreter berbasis stack machine. Program dimulai dengan membaca perintah command line yang diberikan pengguna. Perintah utama berupa file source code Arion berekstensi ".txt", sedangkan perintah tambahan digunakan untuk menentukan file output tertentu seperti output token, parse tree, maupun hasil semantic analysis. Apabila pengguna tidak memberikan file output secara eksplisit, maka hasil proses compiler akan langsung ditampilkan pada terminal.

Setelah validasi argumen selesai dilakukan, program menentukan lokasi file input dan output yang akan digunakan selama proses kompilasi. Program kemudian membuka file

source code dan memulai tahap lexical analysis menggunakan lexer berbasis DFA yang telah diimplementasikan pada milestone sebelumnya. Konfigurasi DFA dimuat dari file konfigurasi lexer sehingga aturan tokenisasi dapat diatur secara fleksibel tanpa perlu mengubah implementasi utama program. Lexer membaca source code karakter demi karakter dan menghasilkan kumpulan token sesuai aturan lexical language Arion. Selama proses pembacaan berlangsung, lexer menerapkan prinsip maximal munch untuk memperoleh lexeme terpanjang yang valid. Setelah seluruh source code selesai diproses, lexer menghasilkan daftar token yang akan diteruskan menuju parser.

Tahap berikutnya adalah syntax analysis menggunakan parser top-down recursive descent. Parser menerima token hasil lexer kemudian melakukan analisis sintaks berdasarkan grammar bahasa Arion yang telah diimplementasikan pada fungsi-fungsi parser. Setiap non-terminal grammar direpresentasikan menggunakan fungsi parsing tersendiri sehingga struktur grammar dapat dipetakan langsung ke dalam implementasi parser. Selama proses parsing berlangsung, parser membangun parse tree yang merepresentasikan struktur sintaks program secara hierarkis. Apabila urutan token tidak sesuai dengan grammar yang berlaku, parser akan menghasilkan syntax error beserta informasi token yang menyebabkan kesalahan. Setelah parsing berhasil dilakukan, parse tree kemudian ditranslasi menjadi Abstract Syntax Tree (AST) agar representasi program menjadi lebih sederhana dan lebih mudah diproses pada tahap semantic analysis.

Tahap semantic analysis dilakukan menggunakan semantic analyzer yang bertugas melakukan dekorasi AST dengan informasi semantic seperti tipe data, scope, symbol reference, dan deklarasi variabel. Semantic analyzer juga membangun symbol table yang digunakan untuk menyimpan informasi identifier selama proses analisis berlangsung. Selain itu, semantic analyzer melakukan pengecekan semantic seperti type checking, undeclared identifier checking, duplicate declaration checking, dan validasi expression. Apabila ditemukan semantic error, pesan error akan ditampilkan sebagai output namun proses semantic analysis tetap dilanjutkan agar seluruh error pada source code dapat terdeteksi sekaligus. Hasil akhir dari tahap semantic analysis berupa Decorated AST dan symbol table yang telah tervalidasi.

Setelah semantic analysis selesai dilakukan, program memasuki tahap backend compiler yang merupakan fokus utama milestone 4. Decorated AST beserta symbol table diteruskan menuju code generator untuk diterjemahkan menjadi intermediate code berbasis stack machine. Pada tahap ini, code generator melakukan traversal AST secara depth-first traversal kemudian menghasilkan instruction program secara sequential. Setiap node AST diterjemahkan menjadi kombinasi instruction tertentu sesuai semantic meaning dari node tersebut. Expression diterjemahkan menggunakan mekanisme stack evaluation, sedangkan statement seperti assignment, if statement, dan while loop diterjemahkan menggunakan kombinasi instruction control flow seperti "JMP" dan "JPC". Instruction hasil translasi kemudian disimpan di dalam instruction program yang merepresentasikan intermediate representation program Arion. Instruction program tersebut selanjutnya diteruskan menuju interpreter untuk dieksekusi menggunakan virtual machine berbasis stack machine.

Interpreter bekerja menggunakan mekanisme fetch-decode-execute cycle. Interpreter membaca instruction satu per satu menggunakan instruction pointer kemudian menjalankan operasi berdasarkan opcode instruction tersebut. Selama runtime execution berlangsung, interpreter memanfaatkan stack machine untuk menyimpan variabel program, activation record, serta evaluation stack yang digunakan untuk evaluasi expression. Pada operasi aritmatika dan relasional, interpreter mengambil operand dari evaluation stack, menjalankan operasi sesuai opcode, kemudian mengembalikan hasil operasi kembali ke stack. Untuk statement control flow seperti if statement dan while loop, interpreter menggunakan instruction jump untuk memindahkan instruction pointer menuju alamat instruction tertentu. Program akan terus menjalankan instruction hingga mencapai instruction "RET" atau seluruh instruction selesai diproses. Setelah runtime execution selesai dilakukan, output program Arion akan ditampilkan pada terminal sebagai hasil akhir proses kompilasi dan interpretasi program.

2.13.2. Kode Program Utama

Implementasi

```
#include <filesystem>
#include <fstream>
#include <iostream>
#include <memory>
#include <sstream>
#include <string>
#include <vector>

#include "backend/code_generator.hpp"
#include "backend/interpreter.hpp"
#include "lexer/dfa.hpp"
#include "lexer/lexer.hpp"
#include "lexer/lexer_exception.hpp"
#include "parser/parser.hpp"
#include "parser/parse_tree.hpp"
#include "semantic/ast_builder.hpp"
#include "semantic/ast_nodes.hpp"
#include "semantic/scope_builder.hpp"
#include "semantic/type_checker.hpp"

namespace fs = std::filesystem;

namespace {

struct Options {
    std::string inputFile;
    std::string saveTokens;
    std::string saveParseTree;
    std::string saveAst;
    std::string outputFile;
    bool lexOnly = false;
    bool verbose = false;
    bool printTac = false;
    bool run = false;
};

void printUsage(const char* programName) {
    std::cout
        << "Usage:\n"
        << "  " << programName << " <program.txt> --lex-only -o <tokens.txt>\n"
        << "  " << programName << " <program.txt> [--verbose]\n"
```

```

        << " " << programName << " <program.txt> --print-tac\n"
        << " " << programName << " <program.txt> --run\n"
        << " " << programName << " <program.txt> --save-tokens <file> [--save-parse-tree
<file>] [--save-ast <file>] [--verbose]\n\n"
        << "Options:\n"
        << " --lex-only          Run lexical analysis only, print tokens, then stop\n"
        << " --verbose          Print tokens, parse tree, and decorated AST to
stdout\n"
        << " --print-tac        Generate TAC, print it, then stop\n"
        << " --run              Generate TAC and run it with the interpreter\n"
        << " --save-tokens <file> Save tokens to <file>\n"
        << " --save-parse-tree <file> Save parse tree to <file>\n"
        << " --save-ast <file>    Save decorated AST, tab, btab, and atab to <file>\n"
        << " -o <file>           Output file. With --lex-only this saves tokens;
otherwise it saves AST\n";
    }

    void ensureParentDirectoryExists(const std::string& filename) {
        fs::path path(filename);
        if (path.has_parent_path()) {
            fs::create_directories(path.parent_path());
        }
    }

    bool isIgnoredByParser(const Token& token) {
        return token.get_type_name() == "COMMENT";
    }

    bool isLexicalErrorToken(const Token& token) {
        return token.get_type_name() == "UNKNOWN";
    }

    void writeOutputFile(const std::string& filename, const std::string& content) {
        ensureParentDirectoryExists(filename);
        std::ofstream out(filename);
        if (!out.is_open()) {
            throw std::runtime_error("Gagal membuka file output: " + filename);
        }
        out << content;
    }

    Options parseOptions(int argc, char* argv[]) {
        Options options;

        if (argc < 2) {
            throw std::runtime_error("Argumen input tidak diberikan");
        }

        int i = 1;
        bool inputSet = false;

        while (i < argc) {
            std::string arg = argv[i];

            if (arg == "--verbose") {
                options.verbose = true;
                ++i;
                continue;
            }

            if (arg == "--print-tac") {
                options.printTac = true;
                ++i;
                continue;
            }

            if (arg == "--run") {
                options.run = true;
                ++i;
                continue;
            }

```

```

    }

    if (arg == "--lex-only") {
        options.lexOnly = true;
        ++i;
        continue;
    }

    if (arg == "--save-tokens") {
        if (i + 1 >= argc) {
            throw std::runtime_error("--save-tokens membutuhkan nama file");
        }
        options.saveTokens = argv[++i];
        ++i;
        continue;
    }

    if (arg == "--save-parse-tree") {
        if (i + 1 >= argc) {
            throw std::runtime_error("--save-parse-tree membutuhkan nama file");
        }
        options.saveParseTree = argv[++i];
        ++i;
        continue;
    }

    if (arg == "--save-ast") {
        if (i + 1 >= argc) {
            throw std::runtime_error("--save-ast membutuhkan nama file");
        }
        options.saveAst = argv[++i];
        ++i;
        continue;
    }

    if (arg == "-o") {
        if (i + 1 >= argc) {
            throw std::runtime_error("-o membutuhkan nama file");
        }
        options.outputFile = argv[++i];
        ++i;
        continue;
    }

    if (!inputSet && arg[0] != '-') {
        options.inputFile = arg;
        inputSet = true;
        ++i;
        continue;
    }

    throw std::runtime_error("Argumen tidak dikenal: " + arg);
}

if (!inputSet) {
    throw std::runtime_error("Argumen input tidak diberikan");
}

if (!options.outputFile.empty()) {
    if (options.lexOnly) {
        options.saveTokens = options.outputFile;
    } else {
        options.saveAst = options.outputFile;
    }
}

return options;
}

void printSemanticResult(std::ostream& out,

```

```

        semantic::AstPtr& astRoot,
        ScopeBuilder& scopeBuilder,
        TypeChecker& typeChecker) {
    semantic::printAst(out, astRoot.get());
    out << "\n";
    scopeBuilder.printTables(out);

    if (scopeBuilder.hasErrors()) {
        out << "\nSemantic errors\n";
        scopeBuilder.printErrors(out);
    }

    if (typeChecker.hasErrors()) {
        out << "\nType errors\n";
        typeChecker.printErrors(out);
    }
}

} // namespace

int main(int argc, char* argv[]) {
    const char* programName = (argc > 0 && argv[0] != nullptr) ? argv[0] : "arion";

    try {
        Options options = parseOptions(argc, argv);

        std::ifstream inputFile(options.inputFile);
        if (!inputFile.is_open()) {
            std::cout << "Gagal membuka file input: " << options.inputFile << "\n";
            return 1;
        }

        std::ostringstream tokenBuffer;
        std::ostringstream parseTreeBuffer;
        std::ostringstream decoratedAstBuffer;

        auto dfa = std::make_shared<DFA>();
        dfa->loadConfig("config/config_lexer.txt");

        Lexer lexer(inputFile, dfa, &tokenBuffer);
        while (!lexer.eof()) {
            lexer.process_next_token();
        }

        if (!options.saveTokens.empty()) {
            writeOutputFile(options.saveTokens, tokenBuffer.str());
        }

        if (options.lexOnly) {
            std::cout << tokenBuffer.str();
            return 0;
        }

        std::vector<Token> parserTokens;
        parserTokens.reserve(lexer.getResult().size());
        for (const Token& token : lexer.getResult()) {
            if (isIgnoredByParser(token)) {
                continue;
            }

            if (isLexicalErrorToken(token)) {
                std::cout << "Lexical error: token tidak dikenal "
                    << token.to_string() << "\n";
                return 1;
            }

            parserTokens.push_back(token);
        }

        Parser parser(parserTokens);

```

```

parse_tree::NodePtr parseRoot = parser.parse();
parse_tree::printTree(parseRoot.get(), parseTreeBuffer);

AstBuilder builder;
semantic::AstPtr astRoot = builder.build(parseRoot);

ScopeBuilder scopeBuilder;
scopeBuilder.build(astRoot);

TypeChecker typeChecker;
typeChecker.check(astRoot.get(), scopeBuilder.symbolTable());

printSemanticResult(decoratedAstBuffer, astRoot, scopeBuilder, typeChecker);

if (options.verbose) {
    std::cout << "=== Lexical analysis ===\n\n";
    std::cout << tokenBuffer.str();
    std::cout << "\n=== Syntax analysis ===\n\n";
    std::cout << parseTreeBuffer.str();
    std::cout << "\n=== Semantic analysis ===\n\n";
    std::cout << decoratedAstBuffer.str();
}

if (!options.saveParseTree.empty()) {
    writeOutputFile(options.saveParseTree, parseTreeBuffer.str());
}

if (!options.saveAst.empty()) {
    writeOutputFile(options.saveAst, decoratedAstBuffer.str());
}

if (options.printTac || options.run) {
    if (scopeBuilder.hasErrors() || typeChecker.hasErrors()) {
        if (scopeBuilder.hasErrors()) {
            std::cout << "Semantic errors\n";
            scopeBuilder.printErrors(std::cout);
        }
        if (typeChecker.hasErrors()) {
            std::cout << "Type errors\n";
            typeChecker.printErrors(std::cout);
        }
        return 1;
    }

    backend::InstructionProgram tac;
    try {
        backend::CodeGenerator codeGenerator;
        tac = codeGenerator.generate(astRoot.get(), scopeBuilder.symbolTable());
    } catch (const std::runtime_error& e) {
        std::cout << e.what() << "\n";
        return 1;
    }

    if (options.printTac) {
        tac.prettyPrint(std::cout);
        return 0;
    }

    try {
        backend::Interpreter interpreter;
        interpreter.execute(tac);
        std::cout << interpreter.getOutput();
    } catch (const std::runtime_error& e) {
        std::cout << e.what() << "\n";
        return 1;
    }

    return 0;
} catch (const LexerException& e) {

```

```

        std::cout << e.full_message() << "\n";
        return 1;
    } catch (const ParseException& e) {
        std::cout << e.full_message() << "\n";
        return 1;
    } catch (const std::runtime_error& e) {
        std::cout << e.what() << "\n";
        printUsage(programName);
        return 1;
    } catch (const std::exception& e) {
        std::cout << "ERROR: " << e.what() << "\n";
        return 1;
    }
}

```

Penjelasan

Kode utama program pada milestone 4 menghubungkan seluruh tahapan compiler, mulai dari lexical analysis, syntax analysis, semantic analysis, hingga backend compiler berupa code generation dan runtime execution menggunakan interpreter stack machine. Program mengimpor berbagai library C++ serta seluruh modul compiler Arion seperti lexer, parser, semantic analyzer, code generator, dan interpreter agar pipeline compiler dapat berjalan terintegrasi.

Program mendefinisikan struktur Options untuk menyimpan konfigurasi command line argument seperti file input, file output, serta mode eksekusi seperti --lex-only, --verbose, --print-tac, dan --run. Fungsi parseOptions() digunakan untuk membaca dan memvalidasi seluruh argument tersebut. Pada fungsi main(), program memproses argument lalu membuka file source code Arion menggunakan std::ifstream. Jika file gagal dibuka, compiler akan menampilkan pesan error dan menghentikan proses.

Tahap lexical analysis dilakukan menggunakan DFA yang dimuat dari file konfigurasi lexer. Lexer membaca source code dan menghasilkan token yang disimpan pada tokenBuffer. Token komentar diabaikan, sedangkan token UNKNOWN digunakan untuk mendeteksi lexical error. Selanjutnya parser recursive descent melakukan syntax analysis dan membangun parse tree berdasarkan grammar bahasa Arion. Parse tree kemudian diterjemahkan menjadi Abstract Syntax Tree (AST) menggunakan AstBuilder. Tahap semantic analysis dilakukan menggunakan kelas ScopeBuilder dan TypeChecker. Kelas ScopeBuilder menambahkan informasi semantic seperti scope dan symbol table, sedangkan kelas TypeChecker memeriksa kesesuaian tipe data pada program. Jika mode --verbose aktif, hasil lexical analysis, parse tree, dan decorated AST akan ditampilkan ke terminal. Program juga dapat menyimpan output tersebut ke file menggunakan writeOutputFile().

Pada tahap backend compiler, kelas CodeGenerator menerjemahkan Decorated AST menjadi intermediate code berbasis stack machine yang disimpan dalam InstructionProgram. Jika mode --print-tac digunakan, intermediate code akan dicetak menggunakan prettyPrint(). Apabila mode --run aktif, interpreter akan menjalankan instruction program menggunakan virtual machine berbasis stack machine. Interpreter mengeksekusi instruction satu per satu dan menampilkan hasil output program ke terminal.

Program juga dilengkapi mekanisme error handling menggunakan blok try-catch untuk menangani berbagai exception seperti lexical error, parse error, semantic error, maupun runtime error agar compiler tetap berjalan secara aman dan informatif.

2.13.3. Kode Code Generator

Header
<pre>#ifndef BACKEND_CODE_GENERATOR_HPP #define BACKEND_CODE_GENERATOR_HPP #include "instruction.hpp" #include "../semantic/ast_nodes.hpp" #include "../semantic/symbol_table.hpp" namespace backend { class CodeGenerator { public: CodeGenerator(); InstructionProgram generate(const semantic::AstNode* root, const semantic::SymbolTable& symbolTable); private: static constexpr int FRAME_HEADER_SIZE = 3; const semantic::SymbolTable* symbolTable_ = nullptr; InstructionProgram program_; int currentLevel_ = 0; void generateNode(const semantic::AstNode* node); void generateStatement(const semantic::AstNode* node); void generateExpression(const semantic::AstNode* node); void generateStore(const semantic::AstNode* target); void generateProgram(const semantic::ProgramNode& node); void generateBlock(const semantic::BlockNode& node); void generateCompound(const semantic::CompoundNode& node); void generateAssign(const semantic::AssignNode& node); void generateProcCall(const semantic::ProcCallNode& node); void generateIf(const semantic::IfNode& node); void generateWhile(const semantic::WhileNode& node); void generateVar(const semantic::VarNode& node); void generateIntLit(const semantic::IntLitNode& node); void generateBoolLit(const semantic::BoolLitNode& node); void generateUnaryOp(const semantic::UnaryOpNode& node); void generateBinOp(const semantic::BinOpNode& node); int variableAddress(const semantic::AstNode& node) const; int levelDifference(const semantic::AstNode& node) const; int initialFrameSize(const semantic::ProgramNode& node) const; static OprCode mapBinaryOp(semantic::BinOpKind op); }; }</pre>
Implementasi
<pre>#include "code_generator.hpp" #include <algorithm> #include <cctype></pre>


```

#include <stdexcept>
#include <string>

namespace backend {

namespace {

std::string unsupported(const semantic::AstNode& node) {
    // Membantu menghasilkan pesan error yang konsisten ketika backend menemukan node AST
    // yang belum didukung
    return std::string("CodeGenerator belum mendukung node ") + node.kindName();
}

bool isExecutableDeclaration(semantic::AstKind kind) {
    return kind == semantic::AstKind::ProcDecl ||
           kind == semantic::AstKind::FuncDecl;
}

std::string lowerName(std::string value) {
    std::transform(value.begin(), value.end(), value.begin(),
        [](unsigned char c) { return static_cast<char>(std::tolower(c)); });
    return value;
}

}

CodeGenerator::CodeGenerator() = default;

InstructionProgram CodeGenerator::generate(const semantic::AstNode* root,
                                           const semantic::SymbolTable& symbolTable) {
    // Titik masuk utama code generation.
    // Input berupa Decorated AST dan Symbol Table hasil Milestone 3, sedangkan output
    // berupa program TAC yang siap diinterpretasikan
    symbolTable_ = &symbolTable;
    program_ = InstructionProgram{};
    currentLevel_ = 0;

    generateNode(root);
    return program_;
}

void CodeGenerator::generateNode(const semantic::AstNode* node) {
    // Dispatcher utama AST
    // Node tingkat atas diterjemahkan ke fungsi generator yang sesuai
    if (!node) return;

    switch (node->getKind()) {
    case semantic::AstKind::Program:
        generateProgram(static_cast<const semantic::ProgramNode&>(*node));
        break;
    case semantic::AstKind::Block:
        generateBlock(static_cast<const semantic::BlockNode&>(*node));
        break;
    case semantic::AstKind::Compound:
        generateCompound(static_cast<const semantic::CompoundNode&>(*node));
        break;
    default:
        generateStatement(node);
        break;
    }
}

void CodeGenerator::generateStatement(const semantic::AstNode* node) {
    // Mengubah statement AST menjadi rangkaian instruksi TAC
    if (!node) return;

    switch (node->getKind()) {
    case semantic::AstKind::Assign:
        generateAssign(static_cast<const semantic::AssignNode&>(*node));
        break;

```

```

        case semantic::AstKind::ProcCall:
            generateProcCall(static_cast<const semantic::ProcCallNode*>(*node));
            break;
        case semantic::AstKind::If:
            generateIf(static_cast<const semantic::IfNode*>(*node));
            break;
        case semantic::AstKind::While:
            generateWhile(static_cast<const semantic::WhileNode*>(*node));
            break;
        case semantic::AstKind::Compound:
            generateCompound(static_cast<const semantic::CompoundNode*>(*node));
            break;
        case semantic::AstKind::DeclPart:
            break;
        case semantic::AstKind::ConstDecl:
        case semantic::AstKind::TypeDecl:
        case semantic::AstKind::VarDecl:
        case semantic::AstKind::SimpleType:
        case semantic::AstKind::ArrayType:
        case semantic::AstKind::RecordType:
        case semantic::AstKind::Range:
        case semantic::AstKind::EnumeratedType:
            break;
        default:
            throw std::runtime_error(unsupported(*node));
    }
}

void CodeGenerator::generateExpression(const semantic::AstNode* node) {
    // Menghasilkan kode evaluasi ekspresi
    // Hasil evaluasi selalu ditempatkan pada evaluation stack
    if (!node) return;

    switch (node->getKind()) {
        case semantic::AstKind::Var:
            generateVar(static_cast<const semantic::VarNode*>(*node));
            break;
        case semantic::AstKind::IntLit:
            generateIntLit(static_cast<const semantic::IntLitNode*>(*node));
            break;
        case semantic::AstKind::BoolLit:
            generateBoolLit(static_cast<const semantic::BoolLitNode*>(*node));
            break;
        case semantic::AstKind::UnaryOp:
            generateUnaryOp(static_cast<const semantic::UnaryOpNode*>(*node));
            break;
        case semantic::AstKind::BinOp:
            generateBinOp(static_cast<const semantic::BinOpNode*>(*node));
            break;
        default:
            throw std::runtime_error(unsupported(*node));
    }
}

void CodeGenerator::generateStore(const semantic::AstNode* target) {
    // Menyimpan hasil ekspresi ke lokasi tujuan assignment
    // Pada Milestone 4 hanya assignment ke variabel biasa yang didukung
    if (!target) return;

    if (target->getKind() != semantic::AstKind::Var) {
        throw std::runtime_error("CodeGenerator fase 1 hanya mendukung assignment ke variabel biasa");
    }

    program_.emit(OpCodes::STO, levelDifference(*target), variableAddress(*target));
}

void CodeGenerator::generateProgram(const semantic::ProgramNode& node) {
    // Menghasilkan TAC untuk program utama
    // Instruksi pertama selalu INT untuk mengalokasikan frame, sedangkan instruksi terakhir

```

```

adalah RET
    currentLevel_ = node.level == semantic::NO_INDEX ? 0 : node.level;

    program_.emit(OpCode::INT, 0, initialFrameSize(node));

    if (node.declaration) {
        for (const auto& declaration : node.declaration->declarations) {
            if (declaration && isExecutableDeclaration(declaration->getKind())) {
                throw std::runtime_error(unsupported(*declaration));
            }
        }
    }

    if (node.statements) {
        generateCompound(*node.statements);
    }
    program_.emit(OpCode::RET, 0, 0);
}

void CodeGenerator::generateBlock(const semantic::BlockNode& node) {
    const int savedLevel = currentLevel_;
    currentLevel_ = node.level == semantic::NO_INDEX ? savedLevel : node.level;
    if (node.statements) generateCompound(*node.statements);
    currentLevel_ = savedLevel;
}

void CodeGenerator::generateCompound(const semantic::CompoundNode& node) {
    // Compound statement diterjemahkan dengan menghasilkan kode untuk setiap statement
    secara berurutan
    for (const auto& statement : node.statements) {
        generateStatement(statement.get());
    }
}

void CodeGenerator::generateAssign(const semantic::AssignNode& node) {
    // Assignment dievaluasi dalam dua tahap:
    // 1. Hitung nilai ekspresi
    // 2. Simpan hasil ke alamat target menggunakan STO
    generateExpression(node.value.get());
    generateStore(node.target.get());
}

void CodeGenerator::generateProcCall(const semantic::ProcCallNode& node) {
    // Milestone 4 hanya mendukung prosedur output bawaan (write/writeln) yang diterjemahkan
    menjadi operasi OPR
    const std::string name = lowerName(node.name);
    const bool isWrite = name == "write";
    const bool isWriteln = name == "writeln";

    if (!isWrite && !isWriteln) {
        throw std::runtime_error(unsupported(node));
    }

    if (node.args.empty()) {
        throw std::runtime_error("CodeGenerator membutuhkan argumen untuk '" + node.name +
            "'");
    }

    for (std::size_t i = 0; i < node.args.size(); ++i) {
        generateExpression(node.args[i].get());
        const bool isLast = i + 1 == node.args.size();
        const OpCode outputOp = (isWriteln && isLast) ? OpCode::WRTLN : OpCode::WRT;
        program_.emit(OpCode::OPR, 0, static_cast<int>(outputOp));
    }
}

void CodeGenerator::generateIf(const semantic::IfNode& node) {
    // IF diterjemahkan menggunakan teknik backpatching:
    //
    // condition

```

```

// JPC <else>
// then-part
// JMP <end>
// else-part
//
// Target lompatan diperbaiki setelah alamat akhir diketahui.
generateExpression(node.condition.get());

const int falseJump = program_.emit(Opcode::JPC, 0, 0);

generateStatement(node.thenStmt.get());

if (node.elseStmt) {
    const int endJump = program_.emit(Opcode::JMP, 0, 0);
    program_.patch(falseJump, program_.currentAddress());
    generateStatement(node.elseStmt.get());
    program_.patch(endJump, program_.currentAddress());
    return;
}

program_.patch(falseJump, program_.currentAddress());
}

void CodeGenerator::generateWhile(const semantic::WhileNode& node) {
    // WHILE diterjemahkan menjadi:
    //
    // loopStart:
    //   condition
    //   JPC exit
    //   body
    //   JMP loopStart
    // exit:
    const int loopStart = program_.currentAddress();

    generateExpression(node.condition.get());
    const int exitJump = program_.emit(Opcode::JPC, 0, 0);

    generateStatement(node.body.get());
    program_.emit(Opcode::JMP, 0, loopStart);
    program_.patch(exitJump, program_.currentAddress());
}

void CodeGenerator::generateVar(const semantic::VarNode& node) {
    // Variabel diterjemahkan menjadi instruksi LOD, sedangkan konstanta diterjemahkan
    // menjadi LIT
    if (!symbolTable_ || node.tabIdx == semantic::NO_INDEX) {
        throw std::runtime_error("variabel '" + node.name + "' belum memiliki entri symbol
table");
    }

    const auto& entry = symbolTable_ -> tabAt(node.tabIdx);
    if (entry.obj == semantic::ObjectKind::Constant) {
        program_.emit(Opcode::LIT, 0, entry.adr);
        return;
    }

    if (entry.obj != semantic::ObjectKind::Variable &&
        entry.obj != semantic::ObjectKind::Parameter) {
        throw std::runtime_error("identifikasi '" + node.name + "' bukan nilai yang dapat
dimuat");
    }

    program_.emit(Opcode::LOD, levelDifference(node), variableAddress(node));
}

void CodeGenerator::generateIntLit(const semantic::IntLitNode& node) {
    program_.emit(Opcode::LIT, 0, static_cast<int>(node.value));
}

void CodeGenerator::generateBoolLit(const semantic::BoolLitNode& node) {

```

```

        program_.emit(Opcode::LIT, 0, node.value ? 1 : 0);
    }

void CodeGenerator::generateUnaryOp(const semantic::UnaryOpNode& node) {
    // Operasi unary diterjemahkan ke opcode TAC yang ekuivalen
    // NOT direalisasikan sebagai perbandingan terhadap nol
    generateExpression(node.operand.get());

    switch (node.op) {
        case semantic::UnaryOpKind::Plus:
            return;
        case semantic::UnaryOpKind::Minus:
            program_.emit(Opcode::OPR, 0, static_cast<int>(OprCode::NEG));
            return;
        case semantic::UnaryOpKind::Not:
            program_.emit(Opcode::LIT, 0, 0);
            program_.emit(Opcode::OPR, 0, static_cast<int>(OprCode::EQL));
            return;
    }
}

void CodeGenerator::generateBinOp(const semantic::BinOpNode& node) {
    // Evaluasi operator biner menggunakan model stack machine dengan operand kiri dan kanan
    // dievaluasi terlebih dahulu, kemudian instruksi OPR dijalankan terhadap keduanya
    generateExpression(node.lhs.get());
    generateExpression(node.rhs.get());

    if (node.op == semantic::BinOpKind::Or) {
        program_.emit(Opcode::OPR, 0, static_cast<int>(OprCode::ADD));
        program_.emit(Opcode::LIT, 0, 0);
        program_.emit(Opcode::OPR, 0, static_cast<int>(OprCode::GTR));
        return;
    }

    program_.emit(Opcode::OPR, 0, static_cast<int>(mapBinaryOp(node.op)));
}

int CodeGenerator::variableAddress(const semantic::AstNode& node) const {
    // Mengubah alamat simbol menjadi alamat runtime
    // FRAME_HEADER_SIZE ditambahkan untuk melewati:
    //
    // 0 = Static Link
    // 1 = Dynamic Link
    // 2 = Return Address
    if (!symbolTable_ || node.tabIdx == semantic::NO_INDEX) {
        throw std::runtime_error("node belum memiliki alamat symbol table");
    }
    return symbolTable_>tabAt(node.tabIdx).adr + FRAME_HEADER_SIZE;
}

int CodeGenerator::levelDifference(const semantic::AstNode& node) const {
    // Menghitung selisih level lexical antara lokasi penggunaan dan lokasi deklarasi
    // identifier
    // Saat ini interpreter hanya menggunakan alamat absolut, tetapi informasi level tetap
    // dihasilkan agar format TAC sesuai spesifikasi
    if (!symbolTable_ || node.tabIdx == semantic::NO_INDEX) {
        return 0;
    }
    const int declarationLevel = symbolTable_>tabAt(node.tabIdx).lev;
    return std::max(0, currentLevel_ - declarationLevel);
}

int CodeGenerator::initialFrameSize(const semantic::ProgramNode& node) const {
    // Menentukan ukuran frame awal program berdasarkan jumlah variabel yang dialokasikan
    // oleh Scope Builder
    if (!symbolTable_) return FRAME_HEADER_SIZE;

    int blockIndex = 0;
    if (node.tabIdx != semantic::NO_INDEX) {
        const auto& programEntry = symbolTable_>tabAt(node.tabIdx);

```

```

        if (programEntry.ref != semantic::NO_INDEX) {
            blockIndex = programEntry.ref;
        }
    }

    return FRAME_HEADER_SIZE + symbolTable_ -> btabAt(blockIndex).vsze;
}

OprCode CodeGenerator::mapBinaryOp(semantic::BinOpKind op) {
    // Memetakan operator AST ke opcode OPR yang ekuivalen
    // Mapping ini mengikuti tabel operasi pada spesifikasi Milestone 4.
    switch (op) {
        case semantic::BinOpKind::Add: return OprCode::ADD;
        case semantic::BinOpKind::Sub: return OprCode::SUB;
        case semantic::BinOpKind::Mul: return OprCode::MUL;
        case semantic::BinOpKind::Div: return OprCode::DIV;
        case semantic::BinOpKind::IntDiv: return OprCode::DIV;
        case semantic::BinOpKind::Mod: return OprCode::MOD;
        case semantic::BinOpKind::Eq: return OprCode::EQL;
        case semantic::BinOpKind::Ne: return OprCode::NEQ;
        case semantic::BinOpKind::Lt: return OprCode::LSS;
        case semantic::BinOpKind::Le: return OprCode::LEQ;
        case semantic::BinOpKind::Gt: return OprCode::GTR;
        case semantic::BinOpKind::Ge: return OprCode::GEQ;
        case semantic::BinOpKind::And:
            return OprCode::MUL;
        case semantic::BinOpKind::Or:
            return OprCode::ADD;
    }
    throw std::runtime_error("operator biner tidak dikenal");
}
}

```

Penjelasan

Kelas CodeGenerator pada milestone 4 berfungsi menerjemahkan Decorated AST menjadi intermediate code berbasis stack machine dalam bentuk InstructionProgram melalui translasi rekursif menggunakan depth-first traversal. Program memanfaatkan beberapa library standar C++ untuk manipulasi string dan penanganan error, sementara helper function internal ditempatkan pada anonymous namespace. Fungsi unsupported() digunakan untuk menghasilkan pesan error bagi node AST yang belum didukung, isExecutableDeclaration() mendeteksi deklarasi prosedur atau fungsi, dan lowerName() mengubah nama menjadi huruf kecil agar pengecekan built-in dapat dilakukan secara case-insensitive. Proses code generation dimulai dari fungsi generate() yang menginisialisasi InstructionProgram dan lexical level sebelum memulai traversal melalui generateNode(). Fungsi generateNode() bertindak sebagai dispatcher AST yang meneruskan translasi ke generator sesuai jenis node seperti program, block, compound, atau statement. Translasi statement dilakukan oleh generateStatement() yang menangani assignment, procedure call, if, while, dan compound statement, sementara deklarasi yang tidak menghasilkan runtime instruction akan dilewati. Untuk expression, generateExpression() menghasilkan instruction evaluasi ke evaluation stack dan mendukung variabel, literal, unary operation, serta binary operation. Penyimpanan hasil expression ke target assignment dilakukan oleh generateStore() menggunakan instruction STO.

Pada level program utama, generateProgram() menghasilkan instruction yang diawali INT untuk alokasi frame dan diakhiri RET, sedangkan generateBlock() menangani perubahan lexical scope dengan memperbarui currentLevel_. Compound statement diterjemahkan secara berurutan oleh generateCompound() menjadi instruction linear, dan assignment diterjemahkan oleh generateAssign() dengan mengevaluasi expression terlebih dahulu sebelum menyimpannya menggunakan STO.

Procedure call diterjemahkan oleh generateProcCall(), khususnya untuk built-in procedure write dan writeln yang menggunakan instruction output OPR. Kontrol alur seperti if dan while diterjemahkan oleh generateIf() dan generateWhile() menggunakan instruction JPC, JMP, serta teknik backpatching. Variabel dan konstanta diterjemahkan oleh generateVar() menggunakan LIT untuk konstanta dan LOD untuk variabel atau parameter, sedangkan literal integer dan boolean diterjemahkan oleh generateIntLit() dan generateBoolLit() menjadi instruction LIT. Unary operation ditangani oleh generateUnaryOp() untuk unary plus, minus (NEG), dan NOT, sementara operasi biner diterjemahkan oleh generateBinOp() menggunakan instruction OPR. Selain itu, variableAddress() bertugas menghitung alamat runtime variabel pada stack machine dan levelDifference() menghitung selisih lexical level. Ukuran frame awal program ditentukan oleh initialFrameSize() untuk instruction INT, dan mapBinaryOp() digunakan untuk memetakan operator AST menjadi opcode OPR yang sesuai.

2.13.4. Kode Instruction

Header
<pre> #ifndef BACKEND_INSTRUCTION_HPP #define BACKEND_INSTRUCTION_HPP #include <iosfwd> #include <string> #include <vector> namespace backend { enum class OpCode { INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET }; const char* toString(OpCode opcode); enum class OprCode { NEG = 1, ADD = 2, SUB = 3, MUL = 4, DIV = 5, MOD = 6, EQL = 7, NEQ = 8, LSS = 9, GEQ = 10, GTR = 11, LEQ = 12, WRT = 13, WRTLN = 14 }; </pre>

```

const char* toString(OprCode opcode);

class Instruction {
public:
    Instruction(OpCode opcode, int level, int operand);

    OpCode opcode;
    int level;
    int operand;
};

class InstructionProgram {
public:
    int emit(OpCode opcode, int level, int operand);
    int emit(const Instruction& instruction);

    void patch(int address, int operand);
    int currentAddress() const;

    const std::vector<Instruction>& getInstructions() const;

    void prettyPrint(std::ostream& out) const;
    std::string prettyPrint() const;

private:
    std::vector<Instruction> instructions_;
};

}

#endif

```

Implementasi

```

#include "instruction.hpp"

#include <ostream>
#include <sstream>
#include <stdexcept>

namespace backend {
// Mengubah opcode TAC menjadi representasi teks agar mudah dibaca saat debugging atau
menggunakan opsi CLI --print-tac
const char* toString(OpCode opcode) {
    // Mengubah sub-operasi OPR menjadi representasi teks
    // Digunakan untuk dokumentasi dan debugging interpreter
    switch (opcode) {
        case OpCode::INT: return "INT";
        case OpCode::LIT: return "LIT";
        case OpCode::LOD: return "LOD";
        case OpCode::STO: return "STO";
        case OpCode::CAL: return "CAL";
        case OpCode::JMP: return "JMP";
        case OpCode::JPC: return "JPC";
        case OpCode::OPR: return "OPR";
        case OpCode::RET: return "RET";
    }
    return "?";
}

const char* toString(OprCode opcode) {
    switch (opcode) {
        case OprCode::NEG: return "NEG";
        case OprCode::ADD: return "ADD";
    }
}

```



```

        case OpCode::SUB: return "SUB";
        case OpCode::MUL: return "MUL";
        case OpCode::DIV: return "DIV";
        case OpCode::MOD: return "MOD";
        case OpCode::EQL: return "EQL";
        case OpCode::NEQ: return "NEQ";
        case OpCode::LSS: return "LSS";
        case OpCode::GEQ: return "GEQ";
        case OpCode::GTR: return "GTR";
        case OpCode::LEQ: return "LEQ";
        case OpCode::WRT: return "WRT";
        case OpCode::WRTLN: return "WRTLN";
    }
    return "?";
}

Instruction::Instruction(OpCode opcode, int level, int operand)
    : opcode(opcode), level(level), operand(operand) {}

int InstructionProgram::emit(OpCode opcode, int level, int operand) {
    // Menambahkan instruksi baru ke program TAC dan mengembalikan alamat instruksi yang
    // baru dibuat
    return emit(Instruction(opcode, level, operand));
}

int InstructionProgram::emit(const Instruction& instruction) {
    // Menyimpan instruksi pada akhir program TAC
    // Nilai return digunakan untuk proses backpatching
    instructions_.push_back(instruction);
    return static_cast<int>(instructions_.size()) - 1;
}

void InstructionProgram::patch(int address, int operand) {
    // Backpatching digunakan ketika target lompatan belum diketahui saat instruksi JMP/JPC
    // dibuat. Setelah alamat tujuan diketahui, operand instruksi diperbarui melalui fungsi ini
    if (address < 0 || address >= static_cast<int>(instructions_.size())) {
        throw std::out_of_range("alamat instruksi untuk patch tidak valid");
    }
    instructions_[static_cast<std::size_t>(address)].operand = operand;
}

int InstructionProgram::currentAddress() const {
    // Mengembalikan alamat instruksi berikutnya yang akan dihasilkan
    // Digunakan oleh CodeGenerator saat menghitung target lompatan
    return static_cast<int>(instructions_.size());
}

const std::vector<Instruction>& InstructionProgram::getInstructions() const {
    return instructions_;
}

void InstructionProgram::prettyPrint(std::ostream& out) const {
    // Menampilkan TAC dalam format:
    // <alamat> <opcode> <level> <operand>
    for (std::size_t i = 0; i < instructions_.size(); ++i) {
        const auto& instruction = instructions_[i];
        out << i << ' '
            << toString(instruction.opcode) << ' '
            << instruction.level << ' '
            << instruction.operand << '\n';
    }
}

std::string InstructionProgram::prettyPrint() const {
    std::ostringstream out;
    prettyPrint(out);
    return out.str();
}

```

Penjelasan

Kelas Instruction pada milestone 4 merepresentasikan intermediate code berbasis stack machine yang digunakan backend compiler dan interpreter. File ini mendefinisikan struktur instruction, opcode, serta container TAC (Three Address Code) hasil CodeGenerator.

Enum OpCode mendefinisikan instruction utama seperti INT, LIT, LOD, STO, JMP, JPC, OPR, dan RET untuk operasi runtime seperti alokasi frame, akses variabel, kontrol alur, operasi aritmatika, dan penghentian program. Sementara itu, enum OprCode digunakan sebagai sub-operasi OPR yang mencakup operasi aritmatika seperti ADD, SUB, MUL, dan DIV, operasi relasional seperti EQL, NEQ, LSS, dan GEQ, serta operasi output seperti WRT dan WRTLN.

Fungsi toString(OpCode) dan toString(OprCode) berfungsi mengubah opcode menjadi representasi teks sehingga TAC lebih mudah dibaca saat proses debugging maupun ketika menggunakan opsi --print-tac. Kelas Instruction sendiri merepresentasikan satu instruction TAC yang terdiri atas tiga komponen utama, yaitu opcode, level, dan operand. opcode menentukan jenis operasi yang dijalankan, level menyimpan lexical level, sedangkan operand menyimpan nilai tambahan seperti alamat memori atau literal.

Kelas InstructionProgram digunakan untuk menyimpan seluruh instruction dalam vector instructions_ dan menyediakan berbagai fungsi utilitas untuk kebutuhan code generation serta debugging. Fungsi emit() digunakan untuk menambahkan instruction baru sekaligus mengembalikan alamat instruction tersebut, yang sangat penting dalam proses backpatching pada struktur kontrol seperti if dan while. Fungsi patch() digunakan untuk memperbarui operand instruction JMP atau JPC setelah target lompatan diketahui.

Selain itu, fungsi currentAddress() mengembalikan alamat instruction berikutnya, sedangkan getInstructions() memberikan akses read-only terhadap seluruh TAC yang telah dihasilkan. Fungsi prettyPrint() mencetak TAC dalam format alamat, opcode, level, dan operand sehingga lebih mudah dianalisis saat debugging. Oleh karena itu, kode Instruction dan kelas utamanya di fungsi InstructionProgram menjadi fondasi utama representasi intermediate code sebelum dijalankan oleh interpreter berbasis stack machine.

2.13.5. Kode Interpreter

Header

```
#ifndef BACKEND_INTERPRETER_HPP
#define BACKEND_INTERPRETER_HPP

#include "instruction.hpp"
#include "stack_machine.hpp"

#include <string>

namespace backend {

class Interpreter {
public:
    void execute(const InstructionProgram& program);
};

}
```

```

        std::string getOutput() const;

private:
    StackMachine machine_;
    std::string output_;

    RuntimeValue executeOpr(int operand);
    RuntimeValue unaryNeg();
    RuntimeValue binaryArithmetic(OprCode opcode);
    RuntimeValue binaryComparison(OprCode opcode);

    int popInteger();
    bool popCondition();
};

}

#endif

```

Implementasi

```

#include "interpreter.hpp"

#include <cstdlib>
#include <stdexcept>

namespace backend {

namespace {

std::size_t checkedAddress(int address, std::size_t limit, const char* context) {
    // Memastikan alamat memori yang digunakan instruksi LOD/STO berada dalam area memori
    runtime yang valid
    if (address < 0 || static_cast<std::size_t>(address) >= limit) {
        throw std::out_of_range(std::string("invalid address: ") + context);
    }
    return static_cast<std::size_t>(address);
}

std::size_t checkedJumpTarget(int address, std::size_t limit) {
    // Memastikan target JMP/JPC menunjuk ke alamat instruksi yang valid sebelum instruction
    pointer diperbarui/ namespace
    if (address < 0 || static_cast<std::size_t>(address) > limit) {
        throw std::out_of_range("invalid address: jump target di luar program");
    }
    return static_cast<std::size_t>(address);
}

}

}

void Interpreter::execute(const InstructionProgram& program) {
    // Setiap eksekusi program dimulai dengan runtime state baru agar tidak ada data yang
    tersisa dari eksekusi sebelumnya
    machine_ = StackMachine{};
    output_.clear();

    const auto& instructions = program.getInstructions();
    std::size_t ip = 0;
    bool running = true;

    while (running && ip < instructions.size()) {
        // Siklus fetch-decode-execute:
        // 1. Ambil instruksi pada IP
        // 2. Decode opcode
        // 3. Jalankan operasi
        // 4. Perbarui IP bila diperlukan
        const Instruction instruction = instructions[ip++];

```

```

        switch (instruction.opcode) {
        case OpCode::INT: // Mengalokasikan area memori runtime sesuai ukuran frame yang
            dihitung saat code generation
            if (instruction.operand < 0) {
                throw std::out_of_range("invalid address: ukuran frame negatif");
            }
            machine_.allocate(static_cast<std::size_t>(instruction.operand));
            break;
        case OpCode::LIT: // Push literal ke evaluation stack
            machine_.push(RuntimeValue::integer(instruction.operand));
            break;
        case OpCode::LOD: // LOD: membaca nilai dari memori runtime ke evaluation stack
            machine_.push(machine_.load(checketAddress(instruction.operand,
                machine_.memorySize(),
                "load")));
            break;
        case OpCode::STO: // STO: menyimpan nilai dari evaluation stack ke memori runtime
            machine_.store(checketAddress(instruction.operand,
                machine_.memorySize(),
                "store"),
                machine_.pop());
            break;
        case OpCode::JMP:
            ip = checkedJumpTarget(instruction.operand, instructions.size());
            break;
        case OpCode::JPC: // JPC melakukan lompatan hanya ketika kondisi bernilai false
            if (!popCondition()) {
                ip = checkedJumpTarget(instruction.operand, instructions.size());
            }
            break;
        case OpCode::OPR:
            executeOpr(instruction.operand);
            break;
        case OpCode::RET:
            running = false;
            break;
        case OpCode::CAL: // CALL procedure/function belum diimplementasikan pada subset
            Milestone 4 yang digunakan interpreter ini
            throw std::runtime_error("invalid opcode: CAL belum didukung interpreter");
        default:
            throw std::runtime_error("invalid opcode");
        }
    }
}

std::string Interpreter::getOutput() const {
    return output_;
}

RuntimeValue Interpreter::executeOpr(int operand) { // Menjalankan seluruh operasi OPR
    sesuai tabel opcode pada spesifikasi Milestone 4
    const OprCode opcode = static_cast<OprCode>(operand);

    switch (opcode) {
    case OprCode::NEG:
        return unaryNeg();
    case OprCode::ADD:
    case OprCode::SUB:
    case OprCode::MUL:
    case OprCode::DIV:
    case OprCode::MOD:
        return binaryArithmetic(opcode); // Operasi aritmetika selalu mengambil dua operand
        dari evaluation stack lalu mendorong hasilnya kembali
    case OprCode::EQL:
    case OprCode::NEQ:
    case OprCode::LSS:
    case OprCode::GEQ:
    case OprCode::GTR:
    case OprCode::LEQ:

```

```

        return binaryComparison(opcode); // Operasi perbandingan menghasilkan RuntimeValue
bertipe Boolean yang kemudian dapat digunakan oleh JPC
    case OprCode::WRT: {
        const RuntimeValue value = machine_.pop();
        output_ += value.toString();
        return value;
    }
    case OprCode::WRTLN: {
        const RuntimeValue value = machine_.pop();
        output_ += value.toString();
        output_ += '\n';
        return value;
    }
    default:
        throw std::runtime_error("invalid opcode: OPR " + std::to_string(opcode));
}

RuntimeValue Interpreter::unaryNeg() {
    const int value = popInteger(); // Boolean direpresentasikan sebagai 0/1 pada runtime,
    sehingga dapat diperlakukan sebagai integer ketika diperlukan
    const RuntimeValue result = RuntimeValue::integer(-value);
    machine_.push(result);
    return result;
}

RuntimeValue Interpreter::binaryArithmetic(OprCode opcode) {
    const int rhs = popInteger();
    const int lhs = popInteger();
    int result = 0;

    switch (opcode) {
    case OprCode::ADD:
        result = lhs + rhs;
        break;
    case OprCode::SUB:
        result = lhs - rhs;
        break;
    case OprCode::MUL:
        result = lhs * rhs;
        break;
    case OprCode::DIV:
        if (rhs == 0) {
            throw std::runtime_error("division by zero");
        }
        result = lhs / rhs;
        break;
    case OprCode::MOD:
        if (rhs == 0) {
            throw std::runtime_error("division by zero");
        }
        result = lhs % rhs;
        break;
    default:
        throw std::runtime_error("invalid opcode");
    }

    const RuntimeValue value = RuntimeValue::integer(result);
    machine_.push(value);
    return value;
}

RuntimeValue Interpreter::binaryComparison(OprCode opcode) {
    const int rhs = popInteger();
    const int lhs = popInteger();
    bool result = false;

    switch (opcode) {
    case OprCode::EQL:
        result = lhs == rhs;

```

```

        break;
    case Opcode::NEQ:
        result = lhs != rhs;
        break;
    case Opcode::LSS:
        result = lhs < rhs;
        break;
    case Opcode::GEQ:
        result = lhs >= rhs;
        break;
    case Opcode::GTR:
        result = lhs > rhs;
        break;
    case Opcode::LEQ:
        result = lhs <= rhs;
        break;
    default:
        throw std::runtime_error("invalid opcode");
}

const RuntimeValue value = RuntimeValue::boolean(result);
machine_.push(value);
return value;
}

int Interpreter::popInteger() {
    const RuntimeValue value = machine_.pop();
    if (value.kind() == RuntimeValue::Kind::Boolean) {
        return value.asBoolean() ? 1 : 0;
    }
    return value.asInteger();
}

bool Interpreter::popCondition() {
    // Mengevaluasi nilai kondisi untuk instruksi JPC
    // Boolean digunakan langsung, sedangkan integer dianggap true jika tidak bernilai nol
    const RuntimeValue value = machine_.pop();
    if (value.kind() == RuntimeValue::Kind::Boolean) {
        return value.asBoolean();
    }
    return value.asInteger() != 0;
}
}

```

Penjelasan

Kelas Interpreter pada milestone 4 berfungsi sebagai virtual machine berbasis stack machine yang menjalankan intermediate code hasil translasi CodeGenerator. Interpreter menerima InstructionProgram berisi instruction TAC (Three Address Code) lalu mengeksekusinya menggunakan mekanisme fetch-decode-execute cycle dengan bantuan StackMachine untuk mengelola evaluation stack dan runtime memory, sedangkan hasil output disimpan pada variabel output_.

Pada file header, kelas Interpreter menyediakan fungsi utama execute() untuk menjalankan program dan getOutput() untuk mengambil hasil output setelah eksekusi selesai. Selain itu, interpreter juga memiliki beberapa helper function seperti executeOpr(), unaryNeg(), binaryArithmetic(), binaryComparison(), popInteger(), dan popCondition() yang digunakan untuk menangani berbagai operasi runtime secara modular sehingga implementasi interpreter menjadi lebih terstruktur dan mudah dipelihara.

Fungsi checkedAddress() digunakan untuk memvalidasi alamat memori pada instruction LOD dan STO, sedangkan checkedJumpTarget() digunakan untuk memvalidasi target lompatan pada instruction

JMP dan JPC. Validasi ini penting untuk mencegah terjadinya invalid memory access maupun runtime error akibat penggunaan alamat instruction yang tidak valid selama proses eksekusi program berlangsung. Kemudian fungsi execute() menjadi inti dari proses runtime execution. Pada awal eksekusi, fungsi ini akan menginisialisasi ulang StackMachine, membersihkan buffer output, kemudian menjalankan instruction menggunakan instruction pointer (ip) melalui mekanisme fetch-decode-execute cycle hingga seluruh program selesai dieksekusi.

Interpreter mendukung beberapa instruction utama, seperti INT untuk alokasi frame runtime, LIT untuk memasukkan literal ke evaluation stack, LOD dan STO untuk akses runtime memory, JMP dan JPC untuk pengendalian alur program, OPR untuk operasi aritmatika, relasional, dan output, serta RET untuk menghentikan eksekusi program. Sementara itu, instruction CAL belum didukung pada milestone ini sehingga penggunaannya akan menghasilkan runtime error.

Terdapat fungsi executeOpr() bertugas menangani seluruh sub-operasi dari instruction OPR dengan memetakan operand ke enum OprCode. Operasi aritmatika seperti ADD, SUB, MUL, DIV, dan MOD diproses melalui fungsi binaryArithmetic(), sedangkan operasi relasional seperti EQL, NEQ, LSS, GEQ, GTR, dan LEQ diproses melalui fungsi binaryComparison(). Selain itu, fungsi ini juga menangani operasi output seperti WRT dan WRTLN. Yang terakhir, fungsi unaryNeg() digunakan untuk menangani unary minus, sedangkan binaryArithmetic() digunakan untuk menjalankan operasi aritmatika sekaligus melakukan pengecekan terhadap kondisi division by zero. Fungsi binaryComparison() digunakan untuk menghasilkan nilai boolean dari operasi relasional. Adapun fungsi popInteger() dan popCondition() digunakan untuk mengambil nilai integer maupun kondisi boolean dari evaluation stack dengan dukungan konversi otomatis antara integer dan boolean agar evaluasi runtime tetap konsisten dan fleksibel.

2.13.6. Kode Stack Machine

Header
<pre> #ifndef BACKEND_STACK_MACHINE_HPP #define BACKEND_STACK_MACHINE_HPP #include <cstdlib> #include <iosfwd> #include <string> #include <vector> namespace backend { // Layout memori runtime: // // 0 : Static Link // 1 : Dynamic Link // 2 : Return Address // 3+ : Variable Area // // Layout ini mengikuti model activation record // yang digunakan pada spesifikasi Milestone 4. class RuntimeValue { public: enum class Kind { </pre>

```

        Integer,
        Boolean
    };

    static RuntimeValue integer(int value);
    static RuntimeValue boolean(bool value);

    Kind kind() const { return kind_; }
    int asInteger() const;
    bool asBoolean() const;

    std::string toString() const;

private:
    RuntimeValue(Kind kind, int value);

    Kind kind_;
    int value_;
};

class StackMachine {
public:
    static constexpr std::size_t MAX_STACK_SIZE = 65536;

    void allocate(std::size_t count);

    void push(RuntimeValue value);
    RuntimeValue pop();
    const RuntimeValue& peek() const;

    RuntimeValue load(std::size_t address) const;
    void store(std::size_t address, RuntimeValue value);

    std::size_t stackSize() const;
    std::size_t memorySize() const;

private:
    static RuntimeValue defaultValue();

    std::vector<RuntimeValue> memory_;
    std::vector<RuntimeValue> stack_;
};

#endif

```

Implementasi

```

#include "stack_machine.hpp"

#include <stdexcept>

namespace backend {

// RuntimeValue merepresentasikan nilai yang dapat disimpan pada stack evaluasi maupun area
memori interpreter
// Milestone 4 saat ini mendukung Integer dan Boolean.
RuntimeValue RuntimeValue::integer(int value) {
    return RuntimeValue(Kind::Integer, value);
}

RuntimeValue RuntimeValue::boolean(bool value) {

```



```

        return RuntimeValue(Kind::Boolean, value ? 1 : 0);
    }

int RuntimeValue::asInteger() const {
    if (kind_ != Kind::Integer) {
        throw std::runtime_error("RuntimeValue bukan integer");
    }
    return value_;
}

bool RuntimeValue::asBoolean() const {
    if (kind_ != Kind::Boolean) {
        throw std::runtime_error("RuntimeValue bukan boolean");
    }
    return value_ != 0;
}

std::string RuntimeValue::toString() const {
    switch (kind_) {
        case Kind::Integer:
            return std::to_string(value_);
        case Kind::Boolean:
            return value_ != 0 ? "true" : "false";
    }
    return "?";
}

RuntimeValue::RuntimeValue(Kind kind, int value)
    : kind_(kind), value_(value) {}

void StackMachine::allocate(std::size_t count) {
    // Mengalokasikan area memori runtime
    // Seluruh slot diinisialisasi dengan nilai default (0)
    memory_.assign(count, defaultValue());
}

void StackMachine::push(RuntimeValue value) {
    if (stack_.size() >= MAX_STACK_SIZE) {
        throw std::runtime_error("stack overflow: ukuran stack melebihi batas runtime");
    }

    stack_.push_back(value);
}

RuntimeValue StackMachine::pop() {
    if (stack_.empty()) {
        throw std::runtime_error("stack underflow: pop dari stack kosong");
    }

    RuntimeValue value = stack_.back();
    stack_.pop_back();
    return value;
}

const RuntimeValue& StackMachine::peek() const {
    if (stack_.empty()) {
        throw std::runtime_error("stack underflow: peek dari stack kosong");
    }

    return stack_.back();
}

RuntimeValue StackMachine::load(std::size_t address) const {
    // LOD pada interpreter membaca nilai dari alamat absolut yang telah ditentukan oleh
    // CodeGenerator
    if (address >= memory_.size()) {
        throw std::out_of_range("invalid memory access: load address " +
            std::to_string(address) +
            " di luar ukuran memory " +
            std::to_string(memory_.size()));
    }
}

```

```

    }

    return memory_[address];
}

void StackMachine::store(std::size_t address, RuntimeValue value) {
    // STO pada interpreter menulis nilai ke alamat memori runtime.
    if (address >= memory_.size()) {
        throw std::out_of_range("invalid memory access: store address " +
                                std::to_string(address) +
                                " di luar ukuran memory " +
                                std::to_string(memory_.size()));
    }

    memory_[address] = value;
}

std::size_t StackMachine::stackSize() const {
    return stack_.size();
}

std::size_t StackMachine::memorySize() const {
    return memory_.size();
}

RuntimeValue StackMachine::defaultValue() {
    // Seluruh variabel diinisialisasi ke integer 0 ketika area memori runtime dibuat
    return RuntimeValue::integer(0);
}

/*
Unit-style contoh penggunaan:

backend::StackMachine machine;
machine.allocate(5); // slot 0..4 tersedia
machine.store(3, backend::RuntimeValue::integer(10));
machine.push(machine.load(3));
machine.push(backend::RuntimeValue::integer(5));
auto rhs = machine.pop();
auto lhs = machine.pop();
machine.store(4, backend::RuntimeValue::integer(lhs.asInteger() + rhs.asInteger()));
*/
}

```

Penjelasan

Kelas StackMachine pada milestone 4 berfungsi sebagai runtime environment yang digunakan interpreter untuk menjalankan intermediate code berbasis stack machine. Komponen ini bertanggung jawab dalam pengelolaan runtime memory, evaluation stack, serta representasi nilai runtime selama proses eksekusi program berlangsung. Selain itu, file ini juga mendefinisikan kelas RuntimeValue yang digunakan untuk merepresentasikan tipe data yang dapat diproses oleh interpreter.

Pada bagian awal file header dijelaskan layout memori runtime yang mengikuti model activation record pada spesifikasi milestone 4. Area memori dibagi menjadi beberapa bagian, yaitu static link, dynamic link, return address, dan variable area. Tiga slot pertama digunakan sebagai header activation record, sedangkan slot berikutnya digunakan untuk menyimpan variabel program. Struktur layout ini digunakan agar runtime memory memiliki organisasi yang konsisten ketika interpreter melakukan akses variabel maupun pengelolaan stack frame.

Kelas RuntimeValue merepresentasikan nilai runtime bertipe integer dan boolean menggunakan enum Kind. Fungsi integer() dan boolean() digunakan untuk membuat objek nilai runtime, sedangkan

asInteger() dan asBoolean() digunakan untuk mengambil nilainya dengan validasi tipe. Fungsi toString() mengubah nilai runtime menjadi string untuk kebutuhan output interpreter. Konstruktor RuntimeValue dibuat private agar pembuatan objek tetap terkontrol melalui fungsi statis.

Kelas StackMachine memiliki dua struktur utama, yaitu memory_ sebagai runtime memory dan stack_ sebagai evaluation stack. Konstanta MAX_STACK_SIZE digunakan untuk membatasi ukuran stack agar terhindar dari stack overflow. Fungsi allocate() digunakan untuk mengalokasikan runtime memory dan menginisialisasi seluruh slot dengan nilai default integer 0 melalui defaultValue(). Fungsi push(), pop(), dan peek() digunakan untuk operasi evaluation stack dengan validasi overflow dan underflow. Fungsi load() dan store() digunakan untuk membaca dan menyimpan nilai pada runtime memory dengan pengecekan validitas alamat memori. Fungsi stackSize() dan memorySize() digunakan untuk memperoleh ukuran stack dan memory saat runtime.

Pada bagian akhir file juga disertakan contoh penggunaan StackMachine dalam bentuk unit-style example. Contoh tersebut memperlihatkan bagaimana runtime memory dialokasikan, bagaimana nilai disimpan menggunakan store(), bagaimana nilai dimuat menggunakan load(), serta bagaimana evaluation stack digunakan untuk melakukan operasi aritmatika sederhana. Contoh ini menunjukkan bahwa StackMachine menjadi fondasi utama runtime execution pada interpreter berbasis stack machine di milestone 4 karena seluruh proses evaluasi expression dan pengelolaan memori dijalankan melalui komponen ini.

BAB III PENGUJIAN

3.1. Pengujian Pertama

Decorated AST (Input1.txt)

```

ProgramNode(name: 'M4Basic')
+-- Declarations
|  \-- VarDecl(name: 'x', type: 'integer')
\-- Block
    +-- Assign
    |  +-- target: Var('x')
    |  \-- value: Num(10)
    \-- ProcedureCall(name: 'writeln', args: [Var('x')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4Basic	38	program	void	0	1	0	0

40	x	39	variable	integer	-1	1	0	0
firstUserIndex = 39								
btab (block table)								
idx	last	lpar	psze	vsze				
---	----	----	----	----				
0	40	-1	0	1				
atab (array table)								
idx	xtyp	etyp	eref	low	high	elsz	size	
---	-----	-----	----	----	----	----	----	
0	unknown	unknown	-1	0	0	1	0	

Three Address Code
0 INT 0 4
1 LIT 0 10
2 STO 0 3
3 LOD 0 3
4 OPR 0 14 ; WRTLN
5 RET 0 0

Output
10

Keterangan
Pengujian pertama ini dilakukan untuk menguji kasus paling dasar berupa deklarasi variabel integer, assignment literal 10 ke variabel x, lalu writeln(x). TAC memakai INT untuk alokasi memori, LIT untuk literal, STO untuk menyimpan, LOD untuk membaca ulang, dan OPR 0 14 untuk writeln.

3.2. Pengujian Kedua

Decorated AST
<pre> ProgramNode(name: 'M4Arithmetic') +-- Declarations +-- VarDecl(name: 'a', type: 'integer') +-- VarDecl(name: 'b', type: 'integer') \-- VarDecl(name: 'c', type: 'integer') \-- Block +-- Assign +-- target: Var('a') \-- value: Num(6) +-- Assign +-- target: Var('b') \-- value: Num(3) +-- Assign +-- target: Var('c') \-- value: BinOp(op: '-') +-- left: BinOp(op: '*') +-- left: BinOp(op: '+') +-- left: Var('a') </pre>

```

|      |      |      \-- right: Var('b')
|      |      |      \-- right: Num(2)
|      |      |      \-- right: BinOp(op: 'DIV')
|      |      |      +-- left: Var('a')
|      |      |      \-- right: Var('b')
\-- ProcedureCall(name: 'writeln', args: [Var('c')])

tab (identifier table)
  idx  identifier      link  obj      type      ref  nrm  lev  adr
  ---  -
  0    -                -1   undefined  unknown   -1   1    0    0
  1    and              -1   reserved  unknown   -1   1    0    0
  2    array            1    reserved  unknown   -1   1    0    0
  3    begin            2    reserved  unknown   -1   1    0    0
  4    case             3    reserved  unknown   -1   1    0    0
  5    const            4    reserved  unknown   -1   1    0    0
  6    div              5    reserved  unknown   -1   1    0    0
  7    downto           6    reserved  unknown   -1   1    0    0
  8    do               7    reserved  unknown   -1   1    0    0
  9    else             8    reserved  unknown   -1   1    0    0
  10   end              9    reserved  unknown   -1   1    0    0
  11   for              10   reserved  unknown   -1   1    0    0
  12   function         11   reserved  unknown   -1   1    0    0
  13   if               12   reserved  unknown   -1   1    0    0
  14   mod              13   reserved  unknown   -1   1    0    0
  15   not              14   reserved  unknown   -1   1    0    0
  16   of               15   reserved  unknown   -1   1    0    0
  17   or               16   reserved  unknown   -1   1    0    0
  18   procedure        17   reserved  unknown   -1   1    0    0
  19   program          18   reserved  unknown   -1   1    0    0
  20   record           19   reserved  unknown   -1   1    0    0
  21   repeat           20   reserved  unknown   -1   1    0    0
  22   then             21   reserved  unknown   -1   1    0    0
  23   to               22   reserved  unknown   -1   1    0    0
  24   type             23   reserved  unknown   -1   1    0    0
  25   until           24   reserved  unknown   -1   1    0    0
  26   var              25   reserved  unknown   -1   1    0    0
  27   while            26   reserved  unknown   -1   1    0    0
  28   integer          27   type      integer   -1   1    0    1
  29   real             28   type      real      -1   1    0    1
  30   char             29   type      char      -1   1    0    1
  31   boolean          30   type      boolean   -1   1    0    1
  32   string           31   type      string    -1   1    0    1
  33   false           32   constant  boolean   -1   1    0    0
  34   true            33   constant  boolean   -1   1    0    1
  35   write           34   procedure  void      -1   1    0    0
  36   writeln         35   procedure  void      -1   1    0    0
  37   read            36   procedure  void      -1   1    0    0
  38   readln          37   procedure  void      -1   1    0    0
  39   M4Arithmetic    38   program   void      0    1    0    0
  40   a               39   variable  integer   -1   1    0    0
  41   b               40   variable  integer   -1   1    0    1
  42   c               41   variable  integer   -1   1    0    2

firstUserIndex = 39

btab (block table)
  idx  last  lpar  psze  vsze
  ---  ---  ---  ---  ---
  0    42    -1    0    3

atab (array table)
  idx  xtyp      etyp      eref  low  high  elsz  size

```

--- 0 unknown unknown --- -1 0 0 --- 1 0 ---
Three Address Code
<pre> 0 INT 0 6 1 LIT 0 6 2 STO 0 3 3 LIT 0 3 4 STO 0 4 5 LOD 0 3 6 LOD 0 4 7 OPR 0 2 ; ADD 8 LIT 0 2 9 OPR 0 4 ; MUL 10 LOD 0 3 11 LOD 0 4 12 OPR 0 5 ; DIV 13 OPR 0 3 ; SUB 14 STO 0 5 15 LOD 0 5 16 OPR 0 14 ; WRTLN 17 RET 0 0 </pre>
Output
16
Keterangan
Pengujian kedua ini ditujukan untuk menguji ekspresi aritmatika bertingkat dengan prioritas operasi: $(a + b) * 2 - a \text{ div } b$. Test ini memastikan generator TAC memecah ekspresi kompleks menjadi operasi stack berurutan: ADD, MUL, DIV, lalu SUB. Dengan $a = 6$ dan $b = 3$, hasilnya $(6 + 3) * 2 - 6 \text{ div } 3 = 18 - 2 = 16$.

3.3. Pengujian Ketiga

Decorated AST
<pre> ProgramNode(name: 'M4IfElse') +-- Declarations +-- VarDecl(name: 'x', type: 'integer') \-- VarDecl(name: 'y', type: 'integer') \-- Block +-- Assign +-- target: Var('x') \-- value: Num(7) +-- If +-- condition: BinOp(op: '>') +-- left: Var('x') \-- right: Num(5) </pre>

```

|   +-- then
|   |   |-- Block
|   |   |   |-- Assign
|   |   |   |   +-- target: Var('y')
|   |   |   |   |-- value: Num(1)
|   |   |-- else
|   |   |   |-- Block
|   |   |   |   |-- Assign
|   |   |   |   |   +-- target: Var('y')
|   |   |   |   |   |-- value: Num(2)
|   |-- ProcedureCall(name: 'writeln', args: [Var('y')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	----	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4IfElse	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0
41	y	40	variable	integer	-1	1	0	1

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----

0	41	-1	0	2				
atab (array table)								
idx	xtyp	etyp	eref	low	high	elsz	size	
---	-----	-----	----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1		
Three Address Code								
0 INT 0 5 1 LIT 0 7 2 STO 0 3 3 LOD 0 3 4 LIT 0 5 5 OPR 0 11 ; GTR 6 JPC 0 10 7 LIT 0 1 8 STO 0 4 9 JMP 0 12 10 LIT 0 2 11 STO 0 4 12 LOD 0 4 13 OPR 0 14 ; WRTLN 14 RET 0 0								
Output								
1								
Keterangan								
Pengujian ketiga ini bertujuan untuk menguji percabangan if-else. Program mengisi x := 7, lalu mengecek x > 5. Karena benar, cabang then dijalankan dan y := 1; cabang else dilewati. TAC menggunakan GTR, JPC untuk lompat jika kondisi salah, dan JMP untuk melewati blok else setelah blok then.								

3.4. Pengujian Keempat

Decorated AST
<pre> ProgramNode(name: 'M4While') +-- Declarations \-- VarDecl(name: 'x', type: 'integer') \-- Block +-- Assign +-- target: Var('x') \-- value: Num(3) \-- While +-- condition: BinOp(op: '>') +-- left: Var('x') \-- right: Num(0) \-- Block +-- ProcedureCall(name: 'writeln', args: [Var('x')]) </pre>

```

\-- Assign
+-- target: Var('x')
\-- value: BinOp(op: '-')
+-- left: Var('x')
\-- right: Num(1)

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4While	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	40	-1	0	1

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Three Address Code
<pre> 0 INT 0 4 1 LIT 0 3 2 STO 0 3 3 LOD 0 3 4 LIT 0 0 5 OPR 0 11 ; GTR 6 JPC 0 14 7 LOD 0 3 8 OPR 0 14 ; WRTLN 9 LOD 0 3 10 LIT 0 1 11 OPR 0 3 ; SUB 12 STO 0 3 13 JMP 0 3 14 RET 0 0 </pre>
Output
<pre> 3 2 1 </pre>
Keterangan
Pengujian keempat ini bertujuan untuk menguji loop while. Program mulai dari $x := 3$, lalu selama $x > 0$, program mencetak x dan mengurangi x dengan 1. TAC menggunakan pola loop: cek kondisi, JPC keluar loop jika salah, body loop, lalu JMP kembali ke kondisi awal.

3.5. Pengujian Kelima

Decorated AST
<pre> ProgramNode(name: 'M4For') +-- Declarations \-- VarDecl(name: 'i', type: 'integer') \-- Block \-- For(var: 'i', start: Num(1), direction: to, end: Num(3)) \-- Block \-- ProcedureCall(name: 'writeln', args: [Var('i')]) </pre>
<pre> tab (identifier table) idx identifier link obj type ref nrm lev adr --- - 0 - -1 undefined unknown -1 1 0 0 1 and -1 reserved unknown -1 1 0 0 2 array 1 reserved unknown -1 1 0 0 3 begin 2 reserved unknown -1 1 0 0 4 case 3 reserved unknown -1 1 0 0 5 const 4 reserved unknown -1 1 0 0 6 div 5 reserved unknown -1 1 0 0 7 downto 6 reserved unknown -1 1 0 0 </pre>

8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4For	38	program	void	0	1	0	0
40	i	39	variable	integer	-1	1	0	0
firstUserIndex = 39								
btab (block table)								
idx	last	lpar	psze	vsze				
---	----	----	----	----				
0	40	-1	0	1				
atab (array table)								
idx	xtyp	etyp	eref	low	high	elsz	size	
---	-----	-----	----	----	----	----	----	
0	unknown	unknown	-1	0	0	1	0	

Three Address Code			
0	INT	0	4
1	LIT	0	1
2	STO	0	3
3	LOD	0	3
4	LIT	0	3
5	OPR	0 12 ; LEQ	
6	JPC	0 14	
7	LOD	0 3	
8	OPR	0 14 ; WRTLN	
9	LOD	0 3	
10	LIT	0 1	
11	OPR	0 2 ; ADD	

```

12 STO 0 3
13 JMP 0 3
14 RET 0 0

```

Output

```

1
2
3

```

Keterangan

Pengujian kelima ini ditujukan untuk menguji translasi for i := 1 to 3 do. Program mencetak nilai i dari 1 sampai 3. TAC menginisialisasi i, mengecek i <= 3 dengan LEQ, menjalankan body, menaikkan i dengan ADD, lalu JMP kembali ke pengecekan. Output: 1, 2, 3.

3.6. Pengujian Keenam

Decorated AST

```

ProgramNode(name: 'M4Repeat')
+-- Declarations
|  \-- VarDecl(name: 'x', type: 'integer')
\-- Block
    +-- Assign
    |  +-- target: Var('x')
    |  \-- value: Num(0)
    \-- Repeat
        +-- Block
        |  +-- Assign
        |  |  +-- target: Var('x')
        |  |  \-- value: BinOp(op: '+')
        |  |      +-- left: Var('x')
        |  |      \-- right: Num(1)
        |  \-- ProcedureCall(name: 'writeln', args: [Var('x')])
        \-- until: BinOp(op: '==')
            +-- left: Var('x')
            \-- right: Num(3)

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0

13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4Repeat	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0

```
firstUserIndex = 39
```

```
btab (block table)
```

idx	last	lpar	psze	vsze
---	----	----	----	----
0	40	-1	0	1

```
atab (array table)
```

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Three Address Code

```
0 INT 0 4
1 LIT 0 0
2 STO 0 3
3 LOD 0 3
4 LIT 0 1
5 OPR 0 2 ; ADD
6 STO 0 3
7 LOD 0 3
8 OPR 0 14 ; WRTLN
9 LOD 0 3
10 LIT 0 3
11 OPR 0 7 ; EQL
12 JPC 0 3
13 RET 0 0
```

Output

1
2
3

Keterangan

Pengujian keenam ini ditujukan untuk menguji repeat-until, yaitu loop yang minimal dijalankan satu kali sebelum kondisi dicek. Program mulai dari $x := 0$, lalu menambah x , mencetaknya, dan berhenti saat $x == 3$. TAC menjalankan body terlebih dahulu, lalu memakai EQL dan JPC untuk kembali ke awal jika kondisi belum terpenuhi.

3.7. Pengujian Ketujuh

Decorated AST

```
ProgramNode(name: 'M4Case')
+-- Declarations
|   \-- VarDecl(name: 'x', type: 'integer')
\-- Block
    +-- Assign
    |   +-- target: Var('x')
    |   \-- value: Num(2)
    \-- Case
        +-- selector: Var('x')
        +-- branch(labels: [Num(1)])
        |   \-- statement
        |       \-- ProcedureCall(name: 'writeln', args: [String('one')])
        +-- branch(labels: [Num(2)])
        |   \-- statement
        |       \-- ProcedureCall(name: 'writeln', args: [String('two')])
        \-- branch(labels: [Num(3)])
            \-- statement
                \-- ProcedureCall(name: 'writeln', args: [String('three')])
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0

20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4Case	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	40	-1	0	1

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Three Address Code									
#STRING 0 "one"									
#STRING 1 "two"									
#STRING 2 "three"									
0	INT	0	4						
1	LIT	0	2						
2	STO	0	3						
3	LOD	0	3						
4	LIT	0	1						
5	OPR	0	7	; EQL					
6	JPC	0	8						
7	JMP	0	9						
8	JMP	0	12						
9	LITS	0	0						
10	OPR	0	14	; WRTLN					
11	JMP	0	30						
12	LOD	0	3						
13	LIT	0	2						
14	OPR	0	7	; EQL					
15	JPC	0	17						
16	JMP	0	18						
17	JMP	0	21						
18	LITS	0	1						


```

19 OPR 0 14 ; WRTLN
20 JMP 0 30
21 LOD 0 3
22 LIT 0 3
23 OPR 0 7 ; EQL
24 JPC 0 26
25 JMP 0 27
26 JMP 0 30
27 LITS 0 2
28 OPR 0 14 ; WRTLN
29 JMP 0 30
30 RET 0 0

```

Output

two

Keterangan

Pengujian ketujuh ini ditujukan untuk menguji statement case dan penggunaan string literal. Program mengisi $x := 2$, lalu memilih cabang yang cocok dari case x of. TAC menyimpan string ke string pool dengan #STRING, lalu membandingkan x dengan tiap label case menggunakan EQL dan JPC/JMP. Karena $x = 2$, cabang kedua dijalankan.

3.8. Pengujian Kedelapan

Decorated AST

```

ProgramNode(name: 'M4Call')
+-- Declarations
|   +-- VarDecl(name: 'y', type: 'integer')
|   +-- ProcedureDecl(name: 'hello', params: [])
|   |   +-- Declarations
|   |   |   \-- Block
|   |   |       \-- ProcedureCall(name: 'writeln', args: [String('hi')])
|   |   \-- FunctionDecl(name: 'add1', params: [integer('x')], returnType:
'integer')
|       +-- Declarations
|       \-- Block
|           \-- Return
|               \-- value: BinOp(op: '+')
|                   +-- left: Var('x')
|                   \-- right: Num(1)
\-- Block
    +-- ProcedureCall(name: 'hello', args: [])
    +-- Assign
    |   +-- target: Var('y')
    |   \-- value: ProcedureCall(name: 'add1')
    |       \-- arg: Num(4)
    \-- ProcedureCall(name: 'writeln', args: [Var('y')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	----	-----	-----	---	---	---	---


```

0 INT 0 4
1 JMP 0 6
2 INT 0 3
3 LITS 0 0
4 OPR 0 14 ; WRTLN
5 RET 0 0
6 JMP 0 14
7 INT 0 3
8 LOD 0 -1
9 LIT 0 1
10 OPR 0 2 ; ADD
11 RET 1 1
12 LIT 0 0
13 RET 1 1
14 CAL 0 2
15 LIT 0 4
16 CAL 0 7
17 STO 0 3
18 LOD 0 3
19 OPR 0 14 ; WRTLN
20 RET 0 0

```

Output

```

Hi
5

```

Keterangan

Pengujian kedelapan ini ditujukan untuk menguji procedure dan function call. Procedure hello mencetak string 'hi', sedangkan function add1(x) mengembalikan $x + 1$. TAC menunjukkan adanya blok procedure/function yang dilompati dulu oleh main program, lalu dipanggil dengan CAL, dan kembali dengan RET. Setelah hello, program menghitung $\text{add1}(4)$ menjadi 5, menyimpannya ke y, lalu mencetaknya.

3.9. Pengujian Kesembilan

Decorated AST

```

ProgramNode(name: 'M4ArrayRecord')
+-- Declarations
|   +-- TypeDecl(name: 'Person', type: 'record')
|   +-- VarDecl(name: 'a', type: 'array[Num(1)..Num(3)] of integer')
|   \-- VarDecl(name: 'p', type: 'Person')
\-- Block
    +-- Assign
    |   +-- target: ArrayAccess
    |   |   +-- base: Var('a')
    |   |   \-- index: Num(1)
    |   \-- value: Num(7)
    +-- Assign

```

```

|   +-- target: ArrayAccess
|   |   +-- base: Var('a')
|   |   \-- index: Num(2)
|   \-- value: BinOp(op: '+')
|       +-- left: ArrayAccess
|       |   +-- base: Var('a')
|       |   \-- index: Num(1)
|       \-- right: Num(5)
+-- Assign
|   +-- target: FieldAccess(field: 'age')
|   |   \-- base: Var('p')
|   \-- value: ArrayAccess
|       +-- base: Var('a')
|       \-- index: Num(2)
\-- ProcedureCall(name: 'writeln', args: [FieldAccess(base: Var('p'),
field: 'age')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4ArrayRecord	38	program	void	0	1	0	0
40	age	-1	field	integer	-1	1	1	0
41	score	40	field	integer	-1	1	1	1

42	Person	39	type	record	1	1	0	2
43	a	42	variable	array	2	1	0	0
44	p	43	variable	record	1	1	0	3
firstUserIndex = 39								
btab (block table)								
idx	last	lpar	psze	vsze				
---	----	----	----	----				
0	44	-1	0	4				
1	41	-1	0	0				
atab (array table)								
idx	xtyp	etyp	eref	low	high	elsz	size	
---	-----	-----	----	----	----	----	----	
0	unknown	unknown	-1	0	0	1	0	
1	integer	integer	-1	1	3	1	3	
2	integer	integer	-1	1	3	1	3	

Three Address Code				
0	INT	0	8	
1	LIT	0	7	
2	ADDR	0	3	
3	LIT	0	1	
4	CHK	1	3	
5	LIT	0	1	
6	OPR	0	3	; SUB
7	LIT	0	1	
8	OPR	0	4	; MUL
9	OPR	0	2	; ADD
10	STOI	0	0	
11	ADDR	0	3	
12	LIT	0	1	
13	CHK	1	3	
14	LIT	0	1	
15	OPR	0	3	; SUB
16	LIT	0	1	
17	OPR	0	4	; MUL
18	OPR	0	2	; ADD
19	LODI	0	0	
20	LIT	0	5	
21	OPR	0	2	; ADD
22	ADDR	0	3	
23	LIT	0	2	
24	CHK	1	3	
25	LIT	0	1	
26	OPR	0	3	; SUB
27	LIT	0	1	
28	OPR	0	4	; MUL
29	OPR	0	2	; ADD
30	STOI	0	0	
31	ADDR	0	3	
32	LIT	0	2	

```

33 CHK 1 3
34 LIT 0 1
35 OPR 0 3 ; SUB
36 LIT 0 1
37 OPR 0 4 ; MUL
38 OPR 0 2 ; ADD
39 LODI 0 0
40 ADDR 0 6
41 STOI 0 0
42 ADDR 0 6
43 LODI 0 0
44 OPR 0 14 ; WRTLN
45 RET 0 0

```

Output

12

Keterangan

Pengujian kesembilan ini ditujukan untuk menguji akses array dan record field. Program membuat array a[1..3] dan record Person dengan field age dan score. Program mengisi a[1] := 7, lalu a[2] := a[1] + 5, kemudian p.age := a[2]. TAC menggunakan instruksi alamat tidak langsung seperti ADDR, LODI, STOI, serta CHK untuk bounds checking array.

3.10. Pengujian Kesepuluh

Decorated AST

```

ProgramNode(name: 'M4DivZero')
+-- Declarations
|  \-- VarDecl(name: 'x', type: 'integer')
\-- Block
+-- Assign
|  +-- target: Var('x')
|  \-- value: BinOp(op: 'DIV')
|     +-- left: Num(1)
|     \-- right: Num(0)
\-- ProcedureCall(name: 'writeln', args: [Var('x')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0

11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4DivZero	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	---	---	---	---
0	40	-1	0	1

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Output

division by zero

Keterangan

Pengujian kesepuluh ini ditujukan untuk menguji runtime error pada interpreter. Program secara semantik valid, tetapi saat dijalankan melakukan 1 div 0. Karena pembagian dengan nol terjadi saat runtime, dimunculkan error dengan pesan “division by zero”. Pengujian ini memastikan interpreter menghentikan eksekusi dan melaporkan error, bukan crash atau menghasilkan nilai salah.

Output
division by zero
Keterangan
Pengujian kesepuluh ini ditujukan untuk menguji runtime error pada interpreter. Program secara semantik valid, tetapi saat dijalankan melakukan 1 div 0. Karena pembagian dengan nol terjadi saat runtime, dimunculkan error dengan pesan “division by zero”. Pengujian ini memastikan interpreter menghentikan eksekusi dan melaporkan error, bukan crash atau menghasilkan nilai salah.

3.11. Pengujian Kesebelas

Decorated AST

```

ProgramNode(name: 'M4ArrayOob')
+-- Declarations
|   +-- VarDecl(name: 'i', type: 'integer')
|   \-- VarDecl(name: 'a', type: 'array[Num(1)..Num(3)] of integer')
\-- Block
    +-- Assign
    |   +-- target: Var('i')
    |   \-- value: Num(4)
    \-- Assign
        +-- target: ArrayAccess
        |   +-- base: Var('a')
        |   \-- index: Var('i')
        \-- value: Num(1)

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4ArrayOob	38	program	void	0	1	0	0
40	i	39	variable	integer	-1	1	0	0
41	a	40	variable	array	2	1	0	1

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	41	-1	0	4

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0
1	integer	integer	-1	1	3	1	3
2	integer	integer	-1	1	3	1	3

Output

ERROR: array index out of bounds: 4 not in [1..3]

Keterangan

Pengujian kesebelas ini dilakukan untuk memeriksa fitur keamanan yang mencegah data menjadi *corrupt* akibat pengaksesan *array* di luar indeks yang dialokasikan. Pada pengujian ini, program berhasil menampilkan pesan *error* dan penyebabnya, sehingga kesalahan ini tidak terjadi saat *runtime*.

3.12. Pengujian Keduabelas

Decorated AST

```
ProgramNode(name: 'M4ForDownto')
+-- Declarations
|   \-- VarDecl(name: 'i', type: 'integer')
\-- Block
    \-- For(var: 'i', start: Num(3), direction: downto, end: Num(1))
        \-- Block
            \-- ProcedureCall(name: 'writeln', args: [Var('i')])
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	----	-----	-----	----	----	----	----
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0

18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4ForDownto	38	program	void	0	1	0	0
40	i	39	variable	integer	-1	1	0	0

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	40	-1	0	1

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Three Address Code

```

0 INT 0 4
1 LIT 0 3
2 STO 0 3
3 LOD 0 3
4 LIT 0 1
5 OPR 0 10 ; GEQ
6 JPC 0 14
7 LOD 0 3
8 OPR 0 14 ; WRTLN
9 LOD 0 3
10 LIT 0 1
11 OPR 0 3 ; SUB
12 STO 0 3
13 JMP 0 3
14 RET 0 0

```

ERROR: invalid address: jump target di luar program

Keterangan

Pengujian keduabelas ini dilakukan untuk memeriksa validasi *code generation* bagian *jump* jika tujuan berada di luar batas yang ditentukan. Pada pengujian ini, program berhasil menampilkan pesan *error* dan penyebabnya, sehingga kesalahan ini tidak terjadi saat *runtime*.

3.13. Pengujian Ketigabelas

Decorated AST

```
ProgramNode(name: 'M4RepeatMore')
+-- Declarations
|  \-- VarDecl(name: 'x', type: 'integer')
\-- Block
+-- Assign
|  +-- target: Var('x')
|  \-- value: Num(1)
\-- Repeat
+-- Block
|  +-- Assign
|  |  +-- target: Var('x')
|  |  \-- value: BinOp(op: '*')
|  |      +-- left: Var('x')
|  |      \-- right: Num(2)
|  \-- ProcedureCall(name: 'writeln', args: [Var('x')])
\-- until: BinOp(op: '>=')
+-- left: Var('x')
\-- right: Num(8)
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0

28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4RepeatMore	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0
firstUserIndex = 39								
btab (block table)								
idx	last	lpar	psze	vsze				
---	----	----	----	----				
0	40	-1	0	1				
atab (array table)								
idx	xtyp	etyp	eref	low	high	elsz	size	
---	-----	-----	----	----	----	----	----	
0	unknown	unknown	-1	0	0	1	0	

Three Address Code													
0	INT	0	4										
1	LIT	0	1										
2	STO	0	3										
3	LOD	0	3										
4	LIT	0	2										
5	OPR	0	4	; MUL									
6	STO	0	3										
7	LOD	0	3										
8	OPR	0	14	; WRTLN									
9	LOD	0	3										
10	LIT	0	8										
11	OPR	0	10	; GEQ									
12	JPC	0	3										
13	RET	0	0										
stack underflow: pop dari stack kosong													
Keterangan													
Pengujian ketigabelas ini dilakukan untuk memeriksa fitur keamanan yang mencegah data menjadi <i>corrupt</i> akibat pengaksesan <i>array</i> di luar indeks yang dialokasikan. Pada pengujian ini, program berhasil menampilkan pesan <i>error</i> dan penyebabnya, sehingga kesalahan ini tidak terjadi saat <i>runtime</i> .													

3.14. Pengujian Keempatbelas

Decorated AST

```

ProgramNode(name: 'M4CaseDefaultless')
+-- Declarations
|  \-- VarDecl(name: 'x', type: 'integer')
\-- Block
    +-- Assign
    |  +-- target: Var('x')
    |  \-- value: Num(3)
    \-- Case
        +-- selector: Var('x')
        +-- branch(labels: [Num(1)])
        |  \-- statement
        |      \-- ProcedureCall(name: 'writeln', args: [String('one')])
        +-- branch(labels: [Num(2)])
        |  \-- statement
        |      \-- ProcedureCall(name: 'writeln', args: [String('two')])
        \-- branch(labels: [Num(3)])
            \-- statement
                \-- ProcedureCall(name: 'writeln', args: [String('three')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	----	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0

38	readln	37	procedure	void	-1	1	0	0
39	M4CaseDefaultless	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0
firstUserIndex = 39								
btab (block table)								
idx	last	lpar	psze	vsze				
---	---	---	---	---				
0	40	-1	0	1				
atab (array table)								
idx	xtyp	etyp	eref	low	high	elsz	size	
---	-----	-----	----	----	----	----	----	
0	unknown	unknown	-1	0	0	1	0	

Three Address Code

```

#STRING 0 "one"
#STRING 1 "two"
#STRING 2 "three"
0 INT 0 4
1 LIT 0 3
2 STO 0 3
3 LOD 0 3
4 LIT 0 1
5 OPR 0 7 ; EQL
6 JPC 0 8
7 JMP 0 9
8 JMP 0 12
9 LITS 0 0
10 OPR 0 14 ; WRTLN
11 JMP 0 30
12 LOD 0 3
13 LIT 0 2
14 OPR 0 7 ; EQL
15 JPC 0 17
16 JMP 0 18
17 JMP 0 21
18 LITS 0 1
19 OPR 0 14 ; WRTLN
20 JMP 0 30
21 LOD 0 3
22 LIT 0 3
23 OPR 0 7 ; EQL
24 JPC 0 26
25 JMP 0 27
26 JMP 0 30
27 LITS 0 2
28 OPR 0 14 ; WRTLN
29 JMP 0 30
30 RET 0 0

```

Keterangan

Pengujian keempatbelas ini dilakukan untuk memeriksa *case statement*. Pada pengujian ini, program berhasil memproses *decorated AST* menjadi TAC tanpa masalah..

3.15. Pengujian Kelimabelas

Decorated AST

```
ProgramNode(name: 'M4FunctionExpression')
+-- Declarations
|   +-- VarDecl(name: 'y', type: 'integer')
|   \-- FunctionDecl(name: 'add1', params: [integer('x')], returnType:
'integer')
|       +-- Declarations
|       \-- Block
|           \-- Return
|               \-- value: BinOp(op: '+')
|                   +-- left: Var('x')
|                   \-- right: Num(1)
\-- Block
    +-- Assign
    |   +-- target: Var('y')
    |   \-- value: BinOp(op: '*')
    |       +-- left: ProcedureCall(name: 'add1')
    |       |   \-- arg: Num(4)
    |       \-- right: Num(2)
    \-- ProcedureCall(name: 'writeln', args: [Var('y')])
```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0

26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4FunctionExpression	38	program	void	0	1	0	0
40	y	39	variable	integer	-1	1	0	0
41	addl	40	function	integer	1	1	0	0
42	x	-1	parameter	integer	-1	1	1	0

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	41	-1	0	1
1	42	42	1	1

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Three Address Code

```

0 INT 0 4
1 JMP 0 9
2 INT 0 3
3 LOD 0 -1
4 LIT 0 1
5 OPR 0 2 ; ADD
6 RET 1 1
7 LIT 0 0
8 RET 1 1
9 LIT 0 4
10 CAL 0 2
11 LIT 0 2
12 OPR 0 4 ; MUL
13 STO 0 3
14 LOD 0 3
15 OPR 0 14 ; WRTLN
16 RET 0 0

```

Keterangan

Pengujian kelimabelas ini dilakukan untuk memeriksa pemanggilan fungsi. Pada pengujian ini, program berhasil menghasilkan *intermediate code* dengan kode *CAL* untuk pemanggilan fungsi *addl*.

3.16. Pengujian Keenambelas

Decorated AST

```

ProgramNode(name: 'M4ProcedureParameter')
+-- Declarations
|   \-- ProcedureDecl(name: 'show', params: [integer('x')])
|       +-- Declarations
|           \-- Block
|               \-- ProcedureCall(name: 'writeln', args: [Var('x')])
\-- Block
    \-- ProcedureCall(name: 'show', args: [Num(9)])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4ProcedureParameter	38	program	void		0	1	0
40	show	39	procedure	void	1	1	0	0
41	x	-1	parameter	integer	-1	1	1	0

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze				
---	----	----	----	----				
0	40	-1	0	0				
1	41	41	1	1				
atab (array table)								
idx	xtyp	etyp	eref	low	high	elsz	size	
---	-----	-----	----	----	----	----	----	
0	unknown	unknown	-1	0	0	1	0	
Three Address Code								
0 INT 0 3 1 JMP 0 6 2 INT 0 3 3 LOD 0 -1 4 OPR 0 14 ; WRTLN 5 RET 1 0 6 LIT 0 9 7 CAL 0 2 8 RET 0 0								
Keterangan								
Pengujian keenambelas ini dilakukan untuk memeriksa pemanggilan prosedur dengan argumen literal. Pada pengujian ini, program berhasil menghasilkan <i>intermediate code</i> dengan kode <i>CAL</i> untuk pemanggilan fungsi <i>show</i> dengan <i>LIT</i> 9.								

3.17. Pengujian Ketujuhbelas

Decorated AST								
ProgramNode (name: 'M4RecordReadback') +-- Declarations +-- TypeDecl (name: 'Point', type: 'record') \-- VarDecl (name: 'p', type: 'Point') \-- Block +-- Assign +-- target: FieldAccess (field: 'x') \-- base: Var ('p') \-- value: Num (4) +-- Assign +-- target: FieldAccess (field: 'y') \-- base: Var ('p') \-- value: BinOp (op: '+') +-- left: FieldAccess (field: 'x') \-- base: Var ('p') \-- right: Num (6) \-- ProcedureCall (name: 'writeln', args: [FieldAccess (base: Var ('p'), field: 'y')])								
tab (identifier table)								
idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	----	-----	-----	---	---	---	---


```

1 LIT 0 4
2 ADDR 0 3
3 STOI 0 0
4 ADDR 0 3
5 LODI 0 0
6 LIT 0 6
7 OPR 0 2 ; ADD
8 ADDR 0 3
9 LIT 0 1
10 OPR 0 2 ; ADD
11 STOI 0 0
12 ADDR 0 3
13 LIT 0 1
14 OPR 0 2 ; ADD
15 LODI 0 0
16 OPR 0 14 ; WRTLN
17 RET 0 0

```

Keterangan

Pengujian ketujuhbelas ini dilakukan untuk memeriksa pemanggilan prosedur *writeln* terhadap nilai yang telah diinisialisasi. Pada pengujian ini, program berhasil menghasilkan *intermediate code* yang tepat.

3.18. Pengujian Kedelapanbelas

Decorated AST

```

ProgramNode(name: 'M4ArrayBoundsValid')
+-- Declarations
|   +-- VarDecl(name: 'i', type: 'integer')
|   \-- VarDecl(name: 'a', type: 'array[Num(1)..Num(3)] of integer')
\-- Block
    +-- Assign
    |   +-- target: Var('i')
    |   \-- value: Num(3)
    +-- Assign
    |   +-- target: ArrayAccess
    |   |   +-- base: Var('a')
    |   |   \-- index: Var('i')
    |   \-- value: Num(12)
    \-- ProcedureCall(name: 'writeln', args: [ArrayAccess(base: Var('a'),
indices: [Var('i')])])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0

7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4ArrayBoundsValid	38	program	void	0	1	0	0
40	i	39	variable	integer	-1	1	0	0
41	a	40	variable	array	2	1	0	1

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	41	-1	0	4

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0
1	integer	integer	-1	1	3	1	3
2	integer	integer	-1	1	3	1	3

Three Address Code

```

0 INT 0 7
1 LIT 0 3
2 STO 0 3
3 LIT 0 12
4 ADDR 0 4
5 LOD 0 3
6 CHK 1 3

```

```

7 LIT 0 1
8 OPR 0 3 ; SUB
9 LIT 0 1
10 OPR 0 4 ; MUL
11 OPR 0 2 ; ADD
12 STOI 0 0
13 ADDR 0 4
14 LOD 0 3
15 CHK 1 3
16 LIT 0 1
17 OPR 0 3 ; SUB
18 LIT 0 1
19 OPR 0 4 ; MUL
20 OPR 0 2 ; ADD
21 LODI 0 0
22 OPR 0 14 ; WRTLN
23 RET 0 0

```

Keterangan

Pengujian ketujuhbelas ini dilakukan untuk memeriksa pengaksesan *array* pada indeks yang valid. Pada pengujian ini, program berhasil menghasilkan *intermediate code* yang tepat tanpa adanya *error*.

3.19. Pengujian Kesembilanbelas

Decorated AST

```

ProgramNode(name: 'M4Readln')
+-- Declarations
|   +-- VarDecl(name: 'x', type: 'integer')
|   +-- VarDecl(name: 'b', type: 'boolean')
|   \-- VarDecl(name: 's', type: 'string')
\-- Block
    +-- ProcedureCall(name: 'readln', args: [Var('x')])
    +-- ProcedureCall(name: 'readln', args: [Var('b')])
    +-- ProcedureCall(name: 'readln', args: [Var('s')])
    +-- ProcedureCall(name: 'writeln', args: [Var('x')])
    +-- ProcedureCall(name: 'writeln', args: [Var('b')])
    \-- ProcedureCall(name: 'writeln', args: [Var('s')])

tab (identifier table)
  idx  identifier      link  obj      type      ref  nrm  lev  adr
  ---  -
  0    -               -1   undefined unknown   -1   1   0   0
  1    and              -1   reserved  unknown   -1   1   0   0
  2    array            1   reserved  unknown   -1   1   0   0
  3    begin            2   reserved  unknown   -1   1   0   0
  4    case             3   reserved  unknown   -1   1   0   0
  5    const            4   reserved  unknown   -1   1   0   0
  6    div              5   reserved  unknown   -1   1   0   0
  7    downto           6   reserved  unknown   -1   1   0   0
  8    do               7   reserved  unknown   -1   1   0   0
  9    else             8   reserved  unknown   -1   1   0   0

```

10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0
26	var	25	reserved	unknown	-1	1	0	0
27	while	26	reserved	unknown	-1	1	0	0
28	integer	27	type	integer	-1	1	0	1
29	real	28	type	real	-1	1	0	1
30	char	29	type	char	-1	1	0	1
31	boolean	30	type	boolean	-1	1	0	1
32	string	31	type	string	-1	1	0	1
33	false	32	constant	boolean	-1	1	0	0
34	true	33	constant	boolean	-1	1	0	1
35	write	34	procedure	void	-1	1	0	0
36	writeln	35	procedure	void	-1	1	0	0
37	read	36	procedure	void	-1	1	0	0
38	readln	37	procedure	void	-1	1	0	0
39	M4Readln	38	program	void	0	1	0	0
40	x	39	variable	integer	-1	1	0	0
41	b	40	variable	boolean	-1	1	0	1
42	s	41	variable	string	-1	1	0	2

firstUserIndex = 39

btab (block table)

idx	last	lpar	psze	vsze
---	----	----	----	----
0	42	-1	0	3

atab (array table)

idx	xtyp	etyp	eref	low	high	elsz	size
---	-----	-----	----	----	----	----	----
0	unknown	unknown	-1	0	0	1	0

Three Address Code

```

0 INT 0 6
1 ADDR 0 3
2 LIT 0 1
3 OPR 0 17 ; RDLN
4 ADDR 0 4
5 LIT 0 4
6 OPR 0 17 ; RDLN
7 ADDR 0 5
8 LIT 0 5
9 OPR 0 17 ; RDLN
10 LOD 0 3

```

```

11 OPR 0 14 ; WRTLN
12 LOD 0 4
13 OPR 0 14 ; WRTLN
14 LOD 0 5
15 OPR 0 14 ; WRTLN
16 RET 0 0

```

Keterangan

Pengujian ketujuhbelas ini dilakukan untuk memeriksa pemanggilan prosedur *readln* dan *writeln*. Pada pengujian ini, program berhasil menghasilkan *intermediate code* yang tepat.

3.20. Pengujian Keduapuluh

Decorated AST

```

ProgramNode(name: 'M4RealArithmetic')
+-- Declarations
|  \-- VarDecl(name: 'x', type: 'real')
\-- Block
    +-- Assign
    |   +-- target: Var('x')
    |   \-- value: BinOp(op: '+')
    |       +-- left: Real(3.5)
    |       \-- right: Real(2)
    \-- ProcedureCall(name: 'writeln', args: [Var('x')])

```

tab (identifier table)

idx	identifier	link	obj	type	ref	nrm	lev	adr
---	-----	---	-----	-----	---	---	---	---
0	-	-1	undefined	unknown	-1	1	0	0
1	and	-1	reserved	unknown	-1	1	0	0
2	array	1	reserved	unknown	-1	1	0	0
3	begin	2	reserved	unknown	-1	1	0	0
4	case	3	reserved	unknown	-1	1	0	0
5	const	4	reserved	unknown	-1	1	0	0
6	div	5	reserved	unknown	-1	1	0	0
7	downto	6	reserved	unknown	-1	1	0	0
8	do	7	reserved	unknown	-1	1	0	0
9	else	8	reserved	unknown	-1	1	0	0
10	end	9	reserved	unknown	-1	1	0	0
11	for	10	reserved	unknown	-1	1	0	0
12	function	11	reserved	unknown	-1	1	0	0
13	if	12	reserved	unknown	-1	1	0	0
14	mod	13	reserved	unknown	-1	1	0	0
15	not	14	reserved	unknown	-1	1	0	0
16	of	15	reserved	unknown	-1	1	0	0
17	or	16	reserved	unknown	-1	1	0	0
18	procedure	17	reserved	unknown	-1	1	0	0
19	program	18	reserved	unknown	-1	1	0	0
20	record	19	reserved	unknown	-1	1	0	0
21	repeat	20	reserved	unknown	-1	1	0	0
22	then	21	reserved	unknown	-1	1	0	0
23	to	22	reserved	unknown	-1	1	0	0
24	type	23	reserved	unknown	-1	1	0	0
25	until	24	reserved	unknown	-1	1	0	0

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Pada milestone 4 ini telah berhasil diimplementasikan backend compiler untuk bahasa Arion yang mampu menerjemahkan Decorated Abstract Syntax Tree (AST) hasil semantic analysis menjadi intermediate code berbasis stack machine dalam bentuk Three Address Code (TAC). Selain itu, sistem juga telah dilengkapi dengan interpreter yang dapat mengeksekusi TAC tersebut sehingga program Arion dapat dijalankan secara langsung pada runtime environment yang telah dibuat.

Proses backend dimulai dari pembentukan TAC menggunakan kelas CodeGenerator. Komponen ini bertugas menerjemahkan berbagai node AST seperti assignment, operasi aritmatika, operasi relasional, percabangan if, perulangan while, serta pemanggilan prosedur bawaan write dan writeln menjadi rangkaian instruction stack machine. Pada tahap ini juga diterapkan mekanisme backpatching untuk menangani target lompatan yang belum diketahui saat proses code generation berlangsung. Intermediate code yang dihasilkan direpresentasikan menggunakan kelas Instruction dan InstructionProgram. Representasi ini mempermudah penyimpanan, manipulasi, serta debugging TAC karena setiap instruction memiliki opcode, lexical level, dan operand yang sesuai dengan spesifikasi milestone 4.

Selain backend compiler, milestone ini juga berhasil mengimplementasikan virtual machine berbasis stack machine melalui kelas Interpreter dan StackMachine. Interpreter menjalankan instruction TAC menggunakan mekanisme fetch-decode-execute cycle serta mendukung berbagai operasi runtime seperti alokasi frame, manipulasi stack, operasi aritmatika, operasi relasional, pengendalian alur program, dan output program. Runtime memory juga telah diorganisasi menggunakan activation record sederhana yang terdiri atas static link, dynamic link, return address, dan variable area. Dari hasil implementasi dan pengujian yang dilakukan, sistem backend dan interpreter telah mampu menjalankan subset bahasa Arion yang digunakan pada milestone 4 dengan baik. Program dapat menghasilkan TAC secara benar serta mengeksekusi hasil translasi tersebut sesuai perilaku program sumber. Dengan demikian, milestone 4 berhasil melengkapi tahapan compiler sebelumnya mulai dari lexical analysis, syntax analysis, semantic analysis, hingga runtime execution..

4.2. Saran

Walaupun backend compiler dan interpreter pada milestone 4 telah berhasil diimplementasikan, masih terdapat beberapa pengembangan yang dapat dilakukan pada tahap berikutnya agar compiler menjadi lebih lengkap dan mendekati implementasi bahasa pemrograman sesungguhnya.

Pengembangan pertama yang dapat dilakukan adalah menambahkan dukungan terhadap procedure dan function call secara penuh melalui implementasi instruction CAL, activation record bertingkat, parameter passing, serta mekanisme return value. Saat ini interpreter masih menggunakan subset sederhana sehingga fitur pemanggilan prosedur dan fungsi belum sepenuhnya didukung. Selain itu, backend juga dapat dikembangkan agar mendukung struktur data yang lebih kompleks seperti array multidimensi, record, pointer, maupun nested scope yang lebih dalam. Dukungan terhadap tipe data tambahan juga dapat memperluas kemampuan bahasa Arion pada tahap runtime execution.

Pengembangan berikutnya adalah melakukan optimisasi intermediate code agar TAC yang dihasilkan menjadi lebih efisien. Contohnya melalui constant folding, dead code elimination, maupun optimisasi penggunaan stack sehingga performa interpreter dapat meningkat.

Dari sisi interpreter, pengembangan dapat dilakukan dengan menambahkan debugging support seperti instruction tracing, stack visualization, maupun runtime inspection agar proses analisis kesalahan program menjadi lebih mudah dilakukan. Terakhir, pengembangan pengujian otomatis juga sangat disarankan agar seluruh komponen compiler dapat divalidasi secara menyeluruh. Dengan adanya automated test untuk lexical analysis, parser, semantic analysis, code generation, dan interpreter, stabilitas sistem dapat lebih terjamin ketika dilakukan penambahan fitur pada milestone berikutnya.

LAMPIRAN

Link GitHub: <https://github.com/alyanrrhma/MAR-Tubes-IF2224-2026>

Tabel Pembagian Tugas:

No.	NIM - Nama	Tugas	Persentase
1.	13524017 - Aziza Dharma Putri	Melakukan penyempurnaan implementasi Milestone 4, termasuk perbaikan backend compiler, intermediate code, interpreter, dan fitur bonus vulnerability. Selain itu, melakukan finalisasi agar implementasi sesuai dengan spesifikasi dan dapat melewati pengujian.	25%
2.	13524053 Ahmad Zaky Robbani	Berkontribusi pada implementasi awal Milestone 4, khususnya pada bagian code generation/interpreter dan sebagian fitur bonus vulnerability. Selain itu, membantu proses debugging dan pengujian hasil akhir program.	25%
3.	13524063 - Marcel Luther Sitorus	Berkontribusi pada proses pengujian Milestone 4, termasuk validasi testcase, pengecekan output interpreter, pengecekan intermediate code, dan pencocokan hasil program dengan expected output.	25%
4.	13524081 - Alya Nur Rahmah	Menyusun laporan Milestone 4, mencakup bagian teori, perancangan, implementasi, kesimpulan, saran, dan dokumentasi laporan. Selain itu, turut mengerjakan implementasi awal Milestone 4 sebelum dilakukan penyempurnaan lebih lanjut oleh anggota lain.	25%

DAFTAR PUSTAKA

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson Education.
- TutorialsPoint. (n.d.). *Compiler Design - Lexical Analysis*. Accessed online from https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson Education. Accessed online from [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)
- Judhi S. (n.d.). *Parse Trees and Conversions*. IF 2224 Teori Bahasa Formal Otomata. Accessed online from https://edunexcontentprodhot.blob.core.windows.net/edunex/2025/81979-Formal-Language-Theory-and-Automata/362559-Module-06/file/1757509600732_slide-08?sv=2024-11-04&spr=https&st=2025-09-10T13%3A06%3A40Z&se=2027-09-10T13%3A06%3A40Z&sr=b&sp=r&sig=BfWy17SqCHFm9Nistwd7qRj0k6k44O53cYS9fUwXqLQ%3D&rscet=application%2Fpdf
- Spivak, R. (2015, December 15). Let's build a simple interpreter. Part 7: Abstract syntax trees. Ruslan's Blog. <https://ruslanspivak.com/lsbasi-part7/>
- Toal, R. (2023). *Compiler Architecture*. Tersedia di: <https://cs.lmu.edu/~ray/notes/compilerarchitecture/>
- GeeksforGeeks. (2024). *Three Address Code in Compiler*. Tersedia di: <https://www.geeksforgeeks.org/three-address-code-compiler/>
- GeeksforGeeks. (2024). *Intermediate Code Generation in Compiler Design*. Tersedia di: <https://www.geeksforgeeks.org/intermediate-code-generation-in-compiler-design/>
- GeeksforGeeks. (2024). *Stack Data Structure*. Tersedia di: <https://www.geeksforgeeks.org/stack-data-structure/>
- Laboratorium Ilmu Rekayasa dan Komputasi ITB. (2026). *Spesifikasi Milestone 4 – Intermediate Code and Interpreter*. IF2224 Teori Bahasa Formal dan Otomata. Institut Teknologi Bandung.